

Playwright with Java

Complete Course Notes

Lectures 0 – 30 | Beginner-Friendly | Interview Ready

[Auto-Wait](#)[Locators](#)[POM](#)[TestNG](#)[Alerts](#)[iFrames](#)[Shadow
DOM](#)[Reports](#)

TABLE OF CONTENTS

Part 1 — Lectures 0–13 — Foundation & Setup

- ◆ Playwright Intro & Architecture
- ◆ Java Fundamentals
- ◆ Maven Setup
- ◆ Exception Handling
- ◆ Java Collections
- ◆ Codegen

Part 2 — Lectures 14–21 — Core Automation

- ◆ BrowserFactory
- ◆ All Locator Types
- ◆ Waits & Navigation
- ◆ Alerts (5 Types)
- ◆ File Upload & Download
- ◆ Dropdowns
- ◆ JavaScript Execution
- ◆ Screenshots & Video

Part 3 — Lectures 22–30 — Advanced Framework

- ◆ Trace Viewer
- ◆ iFrames & Shadow DOM
- ◆ Properties File
- ◆ Apache POI (Excel)
- ◆ Playwright Assertions
- ◆ Extent Reports
- ◆ Page Object Model (POM)
- ◆ TestNG Annotations
- ◆ Cucumber BDD
- ◆ Interview Quick Revision

Playwright with Java — Complete Course Notes (Part 1)

Lectures 0 to 13 | Beginner-Friendly | Interview Ready

How to Use These Notes:
- Read "Definition" first to understand the concept.
- Read "Why We Use It" to understand the purpose.
- Read "Example" to see how it works in code.
- Read "Interview Q&A" before your interview.
- Use "Quick Revision" at the end of each topic for fast review.

LECTURE 0 — Introduction to Playwright

1. What is Playwright?

Definition:

Playwright is a modern tool used to automate web browsers. It was created by Microsoft in 2020. Think of it like a robot that can open a browser, click buttons, fill forms, and check if a website is working correctly — all automatically.

Why We Use It:

- To test websites automatically without clicking manually every time.
- Saves time — runs hundreds of tests in minutes.
- More reliable than Selenium for modern websites.

Key Facts:

- Created by: Microsoft
- Year: 2020
- Type: Open Source (free to use)
- Supported Languages: Java, JavaScript, TypeScript, Python, C#, .NET
- Official Website: <https://playwright.dev/java/docs/intro>

2. Key Features of Playwright

Feature	What It Means in Simple Words
Auto-Wait	Playwright waits for elements to be ready automatically. No need for <code>Thread.sleep()</code> .
Multi-browser	Works with Chrome, Firefox, Safari (WebKit), Edge
Multi-tab	Can handle multiple tabs in one browser
Network Mocking	Can fake/control API responses for testing
Screenshots	Can take pictures of the page automatically
Video Recording	Can record the full test as a video
Tracing	Records every action for debugging
Headless Mode	Can run tests without opening a visible browser

3. Playwright Architecture (How It Works)

Simple Explanation:

Playwright talks to the browser using **WebSocket** — like a phone call that stays connected the whole time.

Selenium used **HTTP** — like sending individual text messages. Every message needs a new connection.

CODE

```
Your Test Code ---(WebSocket)----> Playwright ---> Chrome/Firefox/Safari
```

Why WebSocket is Better:

- One connection for the entire test
- Faster communication
- Less chance of errors
- No need to reconnect for every action

Selenium vs Playwright Architecture:

Point	Selenium	Playwright
Connection Type	HTTP (new request every time)	WebSocket (one connection always open)
Speed	Slower	Faster
Reliability	More flaky	More stable
Auto-wait	No	Yes

4. Playwright Steps (PBP Pattern)

Remember: **P** → **B** → **P**

CODE

```
P = Playwright (entry point)
B = Browser (Chrome/Firefox/WebKit)
P = Page (tab inside browser)
```

JAVA

```
// Step 1: Create Playwright
Playwright pw = Playwright.create();

// Step 2: Launch Browser
Browser browser = pw.chromium().launch();

// Step 3: Create a Tab (Page)
Page page = browser.newPage();

// Step 4: Navigate to URL
page.navigate("https://www.saucedemo.com");
```

Memory Trick: "Please Bring Pizza" = Playwright, Browser, Page

5. First Playwright Program

```
JAVA
package org.sujalkhot;

import com.microsoft.playwright.*;

public class LaunchBrowser {

    public static void main(String[] args) {

        // Creates the Playwright engine – must be first step
        Playwright pw = Playwright.create();

        // Opens Chromium browser with visible UI (headed mode)
        Browser browser = pw.chromium()
            .launch(new BrowserType.LaunchOptions().setHeadless(false));

        // Opens a new tab inside the browser
        Page page = browser.newPage();

        // Goes to the website URL
        page.navigate("https://www.saucedemo.com");

        // Prints the title of the page in console
        System.out.println(page.title());

        // Types "standard_user" in the username field (using CSS ID selector)
        page.fill("#user-name", "standard_user");

        // Types the password
        page.fill("#password", "secret_sauce");

        // Clicks the Login button
        page.click("#login-button");

        // Always close browser at the end
        browser.close();
        pw.close();
    }
}
```

Line by Line Explanation:

- `Playwright.create()` — Starts the Playwright engine. Always first.
 - `setHeadless(false)` — Opens browser with visible window. Default is headless (invisible).
 - `browser.newPage()` — Opens a new tab (like pressing Ctrl+T).
 - `page.navigate(url)` — Types the URL and opens the website.
 - `page.fill("#id", "text")` — Types text into an input field.
 - `page.click("#id")` — Clicks an element.
 - `browser.close()` + `pw.close()` — Cleans up and closes everything.
-

6. Headed vs Headless Mode

Mode	What You See	Speed	When to Use
Headed (<code>setHeadless(false)</code>)	Browser opens visibly	Slower	Debugging, Development
Headless (<code>setHeadless(true)</code>)	No visible browser	Faster	CI/CD, Final Runs

Default: Playwright runs in **Headless** mode by default.

7. Launching Different Browsers

```

JAVA
// Chromium (bundled with Playwright)
Browser browser = pw.chromium().launch();

// Real Google Chrome (must be installed on your computer)
Browser browser = pw.chromium().launch(
    new BrowserType.LaunchOptions().setChannel("chrome"));

// Microsoft Edge
Browser browser = pw.chromium().launch(
    new BrowserType.LaunchOptions().setChannel("msedge"));

// Firefox
Browser browser = pw.firefox().launch();

// WebKit (Safari engine)
Browser browser = pw.webkit().launch();

```

Chrome vs Chromium:

Chrome	Chromium
Made by Google	Open source project
Has Google branding	No branding
Auto-updates	No auto-updates
Installed by user	Bundled with Playwright

8. Maximize Browser Window

```

JAVA
// For Chrome/Edge/Chromium – use argument
Browser browser = pw.chromium().launch(

```



```
new BrowserType.LaunchOptions()
.setHeadless(false)
.setArgs(Arrays.asList("--start-maximized"))
);

// Must set viewport to null for maximized to work
BrowserContext context = browser.newContext(
new Browser.NewContextOptions().setViewportSize(null)
);

Page page = context.newPage();
```

Important: Firefox and WebKit do NOT support `--start-maximized` . For them, set viewport size manually:

```
JAVA
// For Firefox/WebKit fullscreen
context = browser.newContext(
new Browser.NewContextOptions().setViewportSize(1920, 1080)
);
```

9. Interview Questions — Lecture 0

Q1: What is Playwright and who created it?

A: Playwright is a modern web automation framework created by Microsoft in 2020. It supports multiple browsers and multiple programming languages.

Q2: What is the difference between Playwright and Selenium?

A: Playwright uses WebSocket (one persistent connection), while Selenium uses HTTP (new connection for every action). Playwright has built-in auto-waiting, Selenium requires manual waits. Playwright is faster and more stable.

Q3: What languages does Playwright support?

A: Java, JavaScript, TypeScript, Python, C#, and .NET.

Q4: What is the difference between Headless and Headed mode?

A: Headed mode opens a visible browser window. Headless mode runs without any visible window. Headless is faster and used in CI/CD pipelines.

Q5: What is the PBP pattern in Playwright?

A: PBP = Playwright → Browser → Page. First create a Playwright instance, then launch a Browser, then open a Page (tab). This is the mandatory setup order.

LECTURE 1 — Core Concepts & Selenium vs Playwright

1. Default Auto-Wait

Playwright waits **30 seconds** by default for any action to complete before failing.

```
■ JAVA
// To change default timeout for all actions
page.setDefaultTimeout(45000); // 45 seconds

// To change timeout for navigation only
page.setDefaultNavigationTimeout(60000); // 60 seconds
```

2. Selenium vs Playwright Comparison

Feature	Playwright	Selenium
Auto-waiting	Built-in (30 seconds default)	Must write manually
Browser Support	Chromium, Firefox, WebKit	All major browsers
Speed	Faster	Slower
Modern Web Features	Better (single-page apps, shadow DOM, iframes)	Needs workarounds
Installation	Simple Maven dependency	More configuration needed
JavaScript Alerts	<code>page.onDialog()</code>	<code>driver.switchTo().alert()</code>
Multiple Tabs	Built-in	<code>switchTo().window()</code>
Shadow DOM	Auto-piercing	Manual JavaScript workaround
Network Mocking	Built-in	Not available natively

LECTURE 2 — Java Fundamentals for Automation

1. Types of Variables in Java

Variable Type	Where Declared	Example
Local Variable	Inside a method	<code>int x = 5;</code> inside <code>main()</code>
Instance Variable	Inside class, outside method	<code>String name;</code> inside class
Static Variable	Inside class with <code>static</code> keyword	<code>static String browser;</code>

Important: Local variables MUST be initialized before use, or you get a compile error. Instance/static variables get default values (null for String, 0 for int).

2. Common Java Data Types for Automation

■ JAVA

```
String url = "https://example.com"; // Text
int count = 5; // Whole number
boolean isVisible = true; // True or False
long timeout = 30000L; // Large number (note the L at end)
```

LECTURE 3 — Maven Project Setup

1. What is Maven?

Definition: Maven is a build tool that automatically downloads and manages Java libraries (called dependencies or JAR files) for your project.

Without Maven: You download JAR files manually and add them to your project. Very tedious.

With Maven: Just add a few lines to `pom.xml` and Maven downloads everything automatically.

2. Maven Project Structure

■ CODE

```
PlaywrightProject/
■
■ ■ ■ ■ pom.xml ← Add dependencies here
■
■ ■ ■ ■ src/
■ ■ ■ ■ main/
■ ■ ■ ■ java/ ← Main application code
■ ■ ■ ■ resources/ ← Config files (properties, etc.)
■ ■
■ ■ ■ ■ test/
■ ■ ■ ■ java/ ← Your test code goes here
■ ■ ■ ■ resources/ ← Test data (Excel, properties files)
■
■ ■ ■ ■ target/ ← Compiled output (auto-generated)
```

3. Playwright Maven Dependency

Add this to your `pom.xml` inside `<dependencies>` :

■ XML

```
<dependency>
<groupId>com.microsoft.playwright</groupId>
<artifactId>playwright</artifactId>
<version>1.58.0</version>
</dependency>
```

4. How to Add Imports in Eclipse

- Press **Ctrl + Shift + O** to automatically add all needed imports.
 - ■■ Common Mistake: Sometimes Eclipse imports the wrong package. Always double-check the import matches `com.microsoft.playwright` .
-

LECTURE 11 — Exception Handling in Java

1. What is an Exception?

Simple Definition: An exception is an unexpected error that happens while your program is running. Without handling it, your program stops completely.

Real-World Example:

Imagine a test script with 10 steps. If step 5 fails due to an element not found:

- **Without exception handling:** Steps 6–10 never run. Program crashes.
- **With exception handling:** The error is caught, handled, and the remaining steps can continue.

2. Exception Handling Keywords

Keyword	Simple Meaning	When to Use
<code>try</code>	"Try to run this risky code"	Wrap code that might fail
<code>catch</code>	"If it fails, do this instead"	Handle the error
<code>finally</code>	"Always run this no matter what"	Cleanup code (close browser, etc.)
<code>throw</code>	"Manually create an error"	Throw a custom error
<code>throws</code>	"This method might throw an error"	Warn the caller

3. Basic Exception Handling Pattern

```
JAVA
try {
    // Code that might fail
    page.click("#login-btn");
} catch (TimeoutException e) {
    // What to do if it fails
    System.out.println("Element not clickable, trying again...");
    page.click("#login-btn");
} finally {
    // Always runs – perfect for cleanup
    System.out.println("Login action attempted");
    browser.close(); // Always close browser
```

```
}
```

4. Common Exceptions in Automation

Exception	When It Happens
<code>TimeoutException</code>	Element not found within timeout period
<code>NullPointerException</code>	Trying to use an object that is null
<code>NoSuchElementException</code>	Element not found in DOM
<code>ArithmeticException</code>	Dividing by zero
<code>ArrayIndexOutOfBoundsException</code>	Accessing invalid array index

5. Exception Order Rule

Always catch **specific exceptions first**, then general `Exception` :

```
■ JAVA  
try {  
    // code  
} catch (TimeoutException e) { // Specific first  
    // handle timeout  
} catch (Exception e) { // General last  
    // handle anything else  
}
```

■ ■ **Common Mistake:** If you put `catch (Exception e)` first, Java gives a compile error because it would catch everything and the specific catches below it would never run. Always go: Child → Parent.

6. Interview Questions — Exception Handling

Q1: What is the difference between checked and unchecked exceptions?

A: Checked exceptions are caught at compile time (e.g., `IOException` , `FileNotFoundException`). Unchecked exceptions happen at runtime (e.g., `NullPointerException` , `ArithmeticException`).

Q2: Can a try block exist without a catch block?

A: Yes, if a `finally` block is present. `try-finally` is valid.

Q3: Can the finally block be skipped?

A: No, except when `System.exit()` is called.

Q4: Why use specific exceptions instead of just catching `Exception` ?

A: Specific exceptions give more control, better error messages, easier debugging, and avoid accidentally hiding serious errors.

LECTURE 11 (Continued) — Java Collections for Automation

1. What is the Collection Framework?

Simple Definition: Collections are like containers that store groups of data. Instead of creating 10 separate variables, you store all 10 values in one collection.

Memory Trick: Think of collections as different types of boxes:

- **List** = Ordered box (items stay in order, duplicates allowed)
- **Set** = Unique box (no duplicates allowed)
- **Map** = Labeled box (each item has a key and value)

2. Most Important Collections for Automation

ArrayList — Most Used

```
JAVA
// Storing multiple product names
List<String> products = new ArrayList<>();
products.add("Laptop");
products.add("Phone");
products.add("Headphones");
products.add("Laptop"); // Duplicates allowed!

System.out.println(products); // [Laptop, Phone, Headphones, Laptop]
System.out.println(products.get(0)); // Laptop (index-based access)
```

When to use in automation:

- Store all product titles from a page
- Store multiple URLs to test
- Store test data

HashMap — Second Most Used

```
JAVA
// Storing test credentials
Map<String, String> credentials = new HashMap<>();
credentials.put("username", "admin");
credentials.put("password", "admin123");
credentials.put("url", "https://example.com");

System.out.println(credentials.get("username")); // admin
```

When to use in automation:

- Store config data (key-value pairs)
- Store API request parameters
- Store environment-specific values

HashSet — Third Most Used

■ JAVA

```
// Storing unique URLs (no duplicates)
Set<String> urls = new HashSet<>();
urls.add("https://example.com");
urls.add("https://example.com"); // Duplicate - ignored!
urls.add("https://google.com");

System.out.println(urls.size()); // 2 (not 3)
```

When to use in automation:

- Remove duplicate test data
- Store unique browser names
- Verify no duplicate elements on page

3. Collections Comparison Table

Collection	Duplicates	Order	Access	Best For
ArrayList	■ Yes	■ Maintained	By index	Test data lists
LinkedList	■ Yes	■ Maintained	By position	Dynamic data
HashSet	■ No	■ Random	No index	Unique values
LinkedHashSet	■ No	■ Maintained	No index	Unique + ordered
TreeSet	■ No	■ Sorted	No index	Sorted data
HashMap	Keys unique	■ Random	By key	Config/credentials
LinkedHashMap	Keys unique	■ Maintained	By key	Ordered config

4. Iterator — Loop Through Any Collection

■ JAVA

```
ArrayList<String> browsers = new ArrayList<>();
browsers.add("Chrome");
browsers.add("Firefox");
browsers.add("Edge");
```

```
Iterator<String> itr = browsers.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

5. Interview Questions — Collections

Q1: Why is ArrayList used in automation?

A: ArrayList has dynamic size, allows duplicates, maintains insertion order, and supports fast index-based access. Perfect for storing test data, page titles, or product lists.

Q2: Difference between List and Set?

List	Set
Allows duplicates	No duplicates
Ordered (insertion order)	Unordered (HashSet)
Index-based access	No index

Q3: Difference between HashMap and HashSet?

HashMap	HashSet
Stores key-value pairs	Stores only values
Uses <code>put()</code>	Uses <code>add()</code>
Allows one null key	Allows one null value

LECTURE 12 — Playwright Codegen (Recorder)

1. What is Codegen?

Definition: Codegen is Playwright's built-in recorder. When you perform actions on a website manually, Playwright automatically writes the Java code for you.

Real-World Use: You perform a login manually (click, type username, type password, click Login) — Codegen generates all the Playwright Java code automatically.

2. How to Start Codegen

Method 1: Terminal Command

■ **BASH**

```
cd "C:\Your\Playwright\Project\Path"

mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.args="codegen
https://yourwebsite.com"
```

Method 2: Using `page.pause()` in Code (Easier)

■ **JAVA**

```
page.navigate("https://yourwebsite.com");
page.pause(); // Opens Playwright Inspector – record and debug here
```

3. Playwright Inspector Toolbar

Button	Icon	Purpose
Pick Locator	Mouse pointer	Click any element to get its locator
Assert Visibility	Eye 	Check if element is visible
Assert Text	"ab"	Verify text of an element
Assert Value	Textbox	Verify input field value
Record	Red circle	Start/stop recording actions
Inspector	Grid	Explore page elements

4. Generated Code Example

After recording a login:

```
■ JAVA
assertThat(page.getByRole(AriaRole.TEXTBOX,
new Page.GetByRoleOptions().setName("Username"))).isVisible();
page.getByRole(AriaRole.TEXTBOX,
new Page.GetByRoleOptions().setName("Username")).fill("admin");
page.getByRole(AriaRole.TEXTBOX,
new Page.GetByRoleOptions().setName("Password")).fill("admin123");
page.getByRole(AriaRole.BUTTON,
new Page.GetByRoleOptions().setName("Login")).click();
```

5. Interview Questions — Codegen

Q1: What is Playwright Codegen?

A: Codegen is a built-in recorder in Playwright that automatically generates Java/Python/JavaScript code by recording your manual actions on a website. You can start it with the `codegen` CLI command or use `page.pause()` in your script.

Q2: What is `page.pause()` used for?

A: `page.pause()` pauses script execution and opens the Playwright Inspector for debugging, element inspection, and generating locators interactively.

LECTURE 13 — Browser Management

1. Browser Factory Design Pattern (Preview)

Definition: Instead of writing browser launch code in every test, you create one reusable `BrowserFactory` class that handles everything.

Memory Trick: Think of BrowserFactory as a **car rental service**. You tell it what car (browser) you want, and it gives you the car ready to drive.

2. Browser Architecture in Playwright

■ CODE

```
Playwright (main API)
↓
Browser (Chrome/Firefox/WebKit instance)
↓
BrowserContext (like an incognito session)
↓
Page (a tab inside the context)
```

Component	Description	Real-World Analogy
Playwright	Main controller	Remote control
Browser	Running browser	TV
BrowserContext	Isolated session	Separate user profile
Page	One tab	One channel on TV

3. Multiple Browser Contexts

■ JAVA

```
// Two isolated sessions in ONE browser
BrowserContext adminContext = browser.newContext();
BrowserContext userContext = browser.newContext();

Page adminPage = adminContext.newPage();
Page userPage = userContext.newPage();

// Both can be running simultaneously
adminPage.navigate("https://example.com/admin");
```

```
userPage.navigate("https://example.com/user");
```

When to use:

- Test admin and user simultaneously
- Test chat applications (sender and receiver)
- Test multi-user scenarios

4. Interview Questions — Browser Management

Q1: What is BrowserContext in Playwright?

A: BrowserContext is an isolated browser session — like an incognito window. It has its own cookies, localStorage, and session data. Multiple contexts can exist in one browser instance.

Q2: What is the difference between Browser and BrowserContext?

Browser	BrowserContext
Actual running browser	Isolated session inside browser
More memory	Lightweight
Separate browser launch	Created from same browser
Different engine	Multiple sessions

PART 1 — QUICK REVISION BEFORE INTERVIEW

Most Important Points to Remember

■ CODE

1. Playwright = Microsoft, 2020, WebSocket, Auto-Wait 30 seconds
2. PBP Pattern = Playwright → Browser → Page
3. Headless = No visible window (CI/CD)
Headed = Visible window (debugging)
4. setHeadless(false) = opens browser visibly
setHeadless(true) = runs in background
5. Firefox/WebKit = NO --start-maximized
Use setViewportSize(width, height) instead
6. BrowserContext = isolated session (like incognito)
7. Codegen = auto-generates code from your actions
page.pause() = opens inspector for debugging
8. Collection most used in automation:
- ArrayList (test data lists)

- HashMap (config/credentials)
- HashSet (unique values)
9. Exception handling:
try → risky code
catch → handle error
finally → always runs (cleanup)
10. Maven = manages dependencies (JAR files) automatically

Part 1 Complete — Covers Lectures 0 to 13

Continue to Part 2 for Lectures 14 to 20 (Locators, Alerts, JS, Actions)

Playwright with Java — Complete Course Notes (Part 2)

Lectures 14 to 21 | Locators, Alerts, JS, Actions, Screenshots

LECTURE 14 — Browser Factory Design Pattern

1. What is Browser Factory?

Definition: Browser Factory is a design pattern where you write the browser launch code only ONCE in a separate class called **BrowserFactory** , and reuse it everywhere. No need to repeat browser setup in every test.

Real-World Analogy: A remote control factory — you describe what remote (browser) you need, the factory builds it and gives it to you ready to use.

Why We Use It:

- Avoid writing the same browser setup code in 50+ test files
- Easy to change browser settings in one place
- Follows OOP principles (Encapsulation)
- Supports multiple browsers from one method

2. Complete BrowserFactory Code (Corrected)

```
JAVA
package org.sujalkhot.day14;

import java.awt.Dimension;
import java.awt.Toolkit;
import java.util.Arrays;
import com.microsoft.playwright.*;

public class BrowserFactory {
    // private = hidden from outside (Encapsulation)
    // static = shared at class level, no object needed
    private static Playwright playwright;
    private static Browser browser;

    // Call this method to get a Page ready to use
    public static Page initBrowser(String browserName, boolean headless, boolean fullScreen) {
        // Step 1: Start Playwright
        playwright = Playwright.create();

        // Step 2: Get screen size (for Firefox/WebKit fullscreen)
        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int width = (int) screenSize.getWidth();
        int height = (int) screenSize.getHeight();

        // Step 3: Set base options
        BrowserType.LaunchOptions option = new BrowserType.LaunchOptions().setHeadless(headless);

        // Step 4: Add --start-maximized ONLY for Chromium browsers
```

```
if (fullScreen && (browserName.equalsIgnoreCase("Chromium")
|| browserName.equalsIgnoreCase("Chrome")
|| browserName.equalsIgnoreCase("edge")))) {
option.setArgs(Arrays.asList("--start-maximized"));
}

// Step 5: Launch the right browser
switch (browserName.toLowerCase()) {
case "chromium":
browser = playwright.chromium().launch(option);
break;
case "chrome":
option.setChannel("chrome");
browser = playwright.chromium().launch(option);
break;
case "edge":
option.setChannel("edge");
browser = playwright.chromium().launch(option);
break;
case "firefox":
browser = playwright.firefox().launch(option);
break;
case "webkit":
browser = playwright.webkit().launch(option);
break;
default:
throw new IllegalArgumentException("Invalid Browser: " + browserName);
}

// Step 6: Create BrowserContext
BrowserContext context;

if (fullScreen && (browserName.equalsIgnoreCase("chromium")
|| browserName.equalsIgnoreCase("chrome")
|| browserName.equalsIgnoreCase("edge")))) {
// null = let --start-maximized take effect
context = browser.newContext(new Browser.NewContextOptions().setViewportSize(null));
} else if (fullScreen && (browserName.equalsIgnoreCase("firefox")
|| browserName.equalsIgnoreCase("webkit")))) {
// Firefox/WebKit: manually set viewport to screen size
context = browser.newContext(
new Browser.NewContextOptions().setViewportSize(width, height));
} else {
context = browser.newContext(new Browser.NewContextOptions());
}

// Step 7: Open page and return it
return context.newPage();
}
```

```
public static void closeBrowser() {
    if (playwright != null) {
        playwright.close(); // Closes all browsers too
    }
}
}
```

3. Key Points About BrowserFactory

Point	Details
<code>private static</code>	Encapsulation — hidden from outside, shared at class level
<code>playwright.close()</code>	Closes Playwright AND all browsers. Always call this at end.
Firefox/WebKit	Do NOT support <code>--start-maximized</code> . Use <code>setViewportSize()</code> instead.
Chromium fullscreen	Use <code>--start-maximized</code> + <code>setViewportSize(null)</code>
<code>default</code> in switch	Throws error for invalid browser name — "fail fast" approach

4. How to Use BrowserFactory in Tests

```
JAVA
// Get a Page object – browser is launched internally
Page page = BrowserFactory.initBrowser("chrome", false, true);
page.navigate("https://example.com");
// ... your test actions ...

BrowserFactory.closeBrowser(); // Always close at end
```

5. Interview Questions — BrowserFactory

Q1: What is the Browser Factory design pattern?

A: BrowserFactory centralizes browser creation logic in one class. Tests call `initBrowser()` to get a ready Page object without knowing browser setup details. This avoids code duplication and makes maintenance easy.

Q2: Why do Firefox and WebKit not support `--start-maximized` ?

A: `--start-maximized` is a Chromium-specific argument. Firefox and WebKit have different rendering engines and don't accept Chromium flags. Instead, we set viewport size equal to screen dimensions.

Q3: What is Encapsulation in BrowserFactory?

A: The `playwright` and `browser` variables are declared `private` — they cannot be accessed or modified from outside the class. This is encapsulation — hiding internal state.

LECTURE 15 — Locators in Playwright

1. What is a Locator?

Definition: A Locator is like the **GPS address** of a web element. It tells Playwright exactly which element to find and interact with on the page.

Key Properties of Playwright Locators:

- **Auto-wait:** Waits for element to be ready before acting
- **Auto-retry:** Retries if element is not immediately available
- **Re-find:** Re-finds the element before every action
- **Visibility check:** Ensures element is visible before clicking

2. All Locator Types with Examples

Built-in Locators (Use These First — Recommended)

```
■ JAVA
// 1. Role Locator (BEST - ARIA accessibility)
page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Login")).click();
page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Username")).fill("admin");

// 2. Label Locator (for form fields with <label> tags)
page.getByLabel("Email").fill("user@example.com");

// 3. Placeholder Locator (when no ID or label)
page.getByPlaceholder("Enter username").fill("admin");

// 4. Text Locator (finds by visible text)
page.getByText("Login").click();

// 5. Alt Text Locator (for images)
page.getByAltText("Company Logo").isVisible();

// 6. Title Locator (tooltip/title attribute)
page.getByTitle("Close").click();

// 7. Test ID Locator (developer adds data-testid attribute)
page.getByTestId("login-btn").click();
```

CSS Locators

```
■ JAVA
// By ID (#)
page.locator("#username").fill("admin");

// By Class (.)
```

```

page.locator(".login-btn").click();

// By Tag + Attribute
page.locator("input[name='email']").fill("test@test.com");

// By Type
page.locator("input[type='text']").fill("hello");

// CSS starts with, ends with, contains
page.locator("[id^='user']") // ID starts with "user"
page.locator("[id$='name']") // ID ends with "name"
page.locator("[id*='user']") // ID contains "user"

```

XPath Locators

```

■ JAVA
// By ID
page.locator("//input[@id='username']").fill("admin");

// By Text (exact)
page.locator("//button[text()='Login']").click();

// Partial attribute (contains)
page.locator("//input[contains(@id, 'user')]").fill("admin");

// Partial text
page.locator("//button[contains(text(), 'Log')]").click();

// Index-based (XPath index starts at 1!)
page.locator("//input[1]").fill("first input");

// By AND condition
page.locator("//input[@type='text' and @name='email']").fill("test");

// By OR condition
page.locator("//input[@type='file' or @id='fileinput']");

// Wildcard tag (use when tag name might change)
page.locator("//*[@type='file' or @id='fileinput']");

```

Index-Based Locators

```

■ JAVA
// nth() uses 0-based index in Playwright!
page.locator("button").nth(0) // First button
page.locator("button").nth(1) // Second button
page.locator("button").first() // First match
page.locator("button").last() // Last match

```

3. Locator Priority (Use in This Order)

Priority	Locator Type	Why
1	<code>getByRole()</code>	Accessibility-based, most stable
2	<code>getByLabel()</code>	Stable for form fields
3	<code>getByPlaceholder()</code>	Good fallback for inputs
4	<code>getByText()</code>	Human-readable
5	<code>getByTestId()</code>	Most stable in enterprise projects
6	CSS Selector	Flexible, widely used
7	XPath	Use only for complex cases

4. CSS Selector Cheatsheet

Selector	Example	Meaning
<code>#id</code>	<code>#username</code>	By ID ■ Most specific
<code>.class</code>	<code>.login-btn</code>	By class ■
<code>tag[attr='val']</code>	<code>input[name='email']</code>	Tag + attribute ■
<code>div input</code>	<code>div input</code>	Input anywhere inside div
<code>div > input</code>	<code>div > input</code>	Direct child input of div
<code>li:first-child</code>	<code>li:first-child</code>	First list item
<code>li:last-child</code>	<code>li:last-child</code>	Last list item
<code>li:nth-child(2)</code>	<code>li:nth-child(2)</code>	Second list item
<code>[id^='user']</code>	<code>[id^='user']</code>	ID starts with "user"
<code>[id\$='name']</code>	<code>[id\$='name']</code>	ID ends with "name"
<code>[id*='user']</code>	<code>[id*='user']</code>	ID contains "user"

5. ■ CRITICAL: ``.btn primary`` vs ``.btn.primary``

■ JAVA

```
// ■ WRONG – Space means DESCENDANT selector
// Looks for element with class "primary" INSIDE element with class "btn"
page.locator(".btn primary").click(); // Will throw error!

// ■ CORRECT – No space = element must have BOTH classes
```



```
page.locator(".btn.primary").click(); // Correct!
```

Memory Trick: Dot between class names = same element has both classes. Space = parent-child relationship.

6. XPath vs CSS Comparison

Feature	CSS	XPath
Speed	Faster	Slower
Readability	Cleaner	Verbose
Go to parent	■ Cannot	■ Yes (<code>/..</code>)
Sibling access	Limited	■ Full
Text matching	Limited	■ <code>text()</code> and <code>contains()</code>
Recommended	Most cases	Complex DOM only

7. Interview Questions — Locators

Q1: What is the difference between absolute and relative XPath?

A: Absolute XPath starts from the HTML root (`html/body/div/...`) and breaks easily if the page structure changes. Relative XPath starts directly from the target element using `//` and is more stable. Always use relative.

Q2: What is the XPath index vs Playwright `.nth()` index?

A: XPath index starts at **1** — `(//input)[1]` is the first input. Playwright's `.nth()` starts at **0** — `.nth(0)` is the first element. This is a common interview trap!

Q3: Why is `getByRole()` the most preferred locator?

A: It's based on ARIA accessibility roles — the same way screen readers identify elements. It's most stable because it finds elements the way real users see them, not by internal technical attributes.

Q4: What does `data-testid` mean?

A: It's a custom HTML attribute added by developers specifically for automation testing. It doesn't change with UI redesigns, making it the most stable locator for enterprise frameworks.

LECTURE 16 — Role Locators & Element Getter Methods

1. Role Locator Deep Dive

What ARIA Means: ARIA = Accessible Rich Internet Applications. It's a standard that makes web pages readable for people with disabilities. Screen readers use ARIA roles to announce what elements are.

How Playwright finds the element name:

- `aria-label` attribute
- `aria-labelledby` reference
- Associated `<label>` tag
- Inner text of the element
- Placeholder attribute

■ JAVA

```
// Always use setName() to be specific
page.getByRole(AriaRole.TEXTBOX,
new Page.GetByRoleOptions().setName("Username")).fill("admin");
page.getByRole(AriaRole.BUTTON,
new Page.GetByRoleOptions().setName("Login")).click();
page.getByRole(AriaRole.LINK,
new Page.GetByRoleOptions().setName("Logout")).click();
page.getByRole(AriaRole.HEADING,
new Page.GetByRoleOptions().setName("Dashboard"));
page.getByRole(AriaRole.CHECKBOX,
new Page.GetByRoleOptions().setName("Remember me")).check();
```

2. Handling Table Rows with Role Locator

■ JAVA

```
// Find the row containing "John" and click its Edit button
Locator row = page.getByRole(AriaRole.ROW,
new Page.GetByRoleOptions().setName("John"));
row.getByRole(AriaRole.BUTTON,
new Page.GetByRoleOptions().setName("Edit")).click();
```

3. Handling Duplicate Elements Problem

Problem: When multiple elements have the same role + name, Playwright throws an error.

Solutions (in order of preference):

```
■ JAVA
// Solution 1: Use data-testid (BEST)
page.getByTestId("login-password").fill("123456");

// Solution 2: Be more specific with role + name
page.getByRole(AriaRole.TEXTBOX,
new Page.GetByRoleOptions().setName("Password")).fill("12345");

// Solution 3: Use ancestor XPath for product-specific buttons
page.locator("//*[@alt='Isha Test T-Shirt']//ancestor::div//button[@class='btn primary add-btn']").click();

// Solution 4: Use nth() ONLY if UI is fixed (NOT recommended for frameworks)
page.getByPlaceholder("Password").nth(1).fill("1345");
```

4. Page-Level Getter Methods

```
■ JAVA
// Get browser tab title
String title = page.title();

// Get current URL
String url = page.url();

// Get full HTML of the page
String html = page.content();

// Get current viewport size
Page.ViewportSize size = page.viewportSize();

// Get BrowserContext object
BrowserContext ctx = page.context();
```

5. Element-Level Getter Methods

```
■ JAVA
// Get ALL text including hidden text and spaces
String text = page.locator("#element").textContent();

// Get VISIBLE text only (better for user-facing content)
String visibleText = page.locator("#element").innerText();

// Get HTML inside the element
String html = page.locator("#element").innerHTML();

// Get a specific attribute value
String id = page.locator("#element").getAttribute("id");
```

```
String href = page.locator("a").getAttribute("href");

// Get current value of an input field
String value = page.getByPlaceholder("username").inputValue();

// Count how many elements match
int count = page.locator(".product").count();
```

6. textContent() vs innerText() Comparison

Feature	<code>textContent()</code>	<code>innerText()</code>
Includes hidden text	■ Yes	■ No
Includes extra spaces	■ Yes	■ No
What it returns	All raw text	Only visible text
When to use	Reading raw DOM content	Checking what user sees

Best Practice: Always use `.trim()` after `textContent()` in real projects:

```
■ JAVA
// Remove leading/trailing spaces
String label = page.locator("//label[1]").textContent().trim();
```

7. Interview Questions — Role & Getters

Q1: What is ARIA and why does it matter for Playwright?

A: ARIA (Accessible Rich Internet Applications) adds accessibility metadata to HTML. Playwright's `getByRole()` uses ARIA roles to find elements — the same way screen readers identify them. This makes tests more aligned with real user experience.

Q2: What does `locator.count()` return if no elements are found?

A: It returns `0`. Unlike `.nth()` or `.click()`, it does NOT throw an error, making it safe to check if elements exist.

LECTURE 17 — Navigation, Waits, and Form Actions

1. Navigation Commands

```
JAVA
// Open a URL
page.navigate("https://example.com");

// Navigate with custom wait strategy
page.navigate("https://example.com",
new Page.NavigateOptions().setWaitUntil(WaitUntilState.DOMCONTENTLOADED));

// Refresh the page
page.reload();

// Go back
page.goBack();

// Go forward
page.goForward();

// Wait for navigation after clicking a link
page.waitForNavigation(() -> {
page.locator("text=Login").click(); // Action that causes navigation
});
```

2. Page Load States

State	Waits For	Speed	Use When
DOMCONTENTLOADED	HTML parsed only	Fastest	Page structure is enough
LOAD	HTML + images + CSS	Medium	Normal pages
NETWORKIDLE	All API calls done	Slowest	React/Angular/Vue pages

```
JAVA
page.waitForLoadState(LoadState.DOMCONTENTLOADED); // Fast
page.waitForLoadState(LoadState.LOAD); // Normal
page.waitForLoadState(LoadState.NETWORKIDLE); // API pages
```

Memory Trick:

```
CODE
DOMCONTENTLOADED = HTML skeleton loaded
LOAD = Full body loaded (HTML + images + CSS)
```

```
NETWORKIDLE = All data loaded (API calls done)
```

3. Auto-Waiting vs Explicit Waits

Auto-Waiting (Built-in — Use First)

Playwright automatically checks before EVERY action:

- ■ Element exists in DOM
- ■ Element is visible
- ■ Element is enabled
- ■ Element is stable (not animating)

■ JAVA

```
// These all have auto-waiting built in:
page.click("#submit"); // Auto-waits
page.fill("#username", "admin"); // Auto-waits
page.check("#checkbox"); // Auto-waits
page.hover(".menu"); // Auto-waits
page.selectOption("#country", "India"); // Auto-waits
```

Explicit Waits (Use When Auto-Wait is Not Enough)

■ JAVA

```
// Wait for element to appear
page.waitForSelector("#success-message");

// Wait with custom timeout (overrides 30s default)
page.waitForSelector("#success-message",
    new Page.WaitForSelectorOptions().setTimeout(5000));

// Wait for element to become visible
page.locator("#submit")
    .waitFor(new Locator.WaitForOptions()
        .setState(WaitForSelectorState.VISIBLE));

// Wait for loader to disappear
page.waitForSelector(".spinner",
    new Page.WaitForSelectorOptions()
        .setState(WaitForSelectorState.HIDDEN));

// Wait for text to change
page.waitForFunction(
    "() => document.querySelector('#status').innerText === 'Completed'");
```

4. When to Use Auto-Wait vs Explicit Wait

Scenario	Best Choice
Button click	■ Auto-Wait
Fill input field	■ Auto-Wait
Check checkbox	■ Auto-Wait
Select dropdown	■ Auto-Wait
Wait for loader to disappear	■ Explicit (HIDDEN state)
API response loaded	■ Explicit (NETWORKIDLE)
Text changes dynamically	■ Explicit (waitForFunction)

5. Why NOT Thread.sleep()

■ JAVA

```
// ■ NEVER use this in Playwright
Thread.sleep(5000);
```

Problems:

- Always waits full 5 seconds even if element loads in 1 second
- Makes tests unnecessarily slow
- If element takes 6 seconds, test still fails
- Not reliable for dynamic content

■ JAVA

```
// ■ Use Playwright's built-in waiting instead
page.locator("#submit").waitFor(); // Waits until ready
```

6. Global Timeout Settings

■ JAVA

```
// Set timeout for ALL actions (click, fill, check, etc.)
page.setDefaultTimeout(10000); // 10 seconds

// Set timeout ONLY for page navigation
page.setDefaultNavigationTimeout(15000); // 15 seconds
```

7. fill() vs type()

Feature	<code>fill()</code>	<code>type()</code>
Clears existing text	■ Yes	■ No
Appends to existing	■ No	■ Yes
Speed	Fast (one operation)	Slower (char by char)
Use case	Most automation	Special typing behavior

■ JAVA

```
// fill() – clears field and types new value
page.fill("#username", "admin");
page.locator("#username").fill("admin");

// Clear a field
page.fill("#email", "");

// type() – appends without clearing
// If field has "Hello" and you type " World" → "Hello World"
page.type("#search", " World");

// Get current value of input
String value = page.inputValue("#username");

// Clear using clear()
page.locator("#username").clear();
```

8. Radio Buttons and Checkboxes

■ JAVA

```
// Select a radio button
page.check("input[value='male']");
page.check("#gender-male");

// Verify radio button is selected
boolean isSelected = page.isChecked("#gender-male");

// Checkbox check
page.getByRole(AriaRole.CHECKBOX,
new Page.GetByRoleOptions().setName("Remember me")).check();

// Uncheck
page.uncheck("#remember");
```

9. Date Picker

■ JAVA

```
// Fill date picker directly (format: YYYY-MM-DD)
```



```
page.fill("#datePicker", "2025-10-10");  
  
// Using JavaScript if the field is readonly  
page.evaluate("document.querySelector('#datePicker').value = '2026-01-15'");
```

10. Interview Questions — Navigation & Waits

Q1: What is the difference between `DOMCONTENTLOADED` , `LOAD` , and `NETWORKIDLE` ?

A: `DOMCONTENTLOADED` waits only for HTML parsing. `LOAD` waits for HTML + images + CSS. `NETWORKIDLE` waits until all network requests stop. Use `LOAD` for most cases and `NETWORKIDLE` for API-driven pages.

Q2: Why is `fill()` preferred over `type()` ?

A: `fill()` clears existing text and enters new value in one fast operation. `type()` simulates keyboard typing character by character without clearing. Use `fill()` for most scenarios.

Q3: What is the Retry Mechanism in Playwright?

A: Playwright automatically retries actions until the element becomes ready or the timeout (default 30s) is reached. No manual retry loops needed. Locators re-check the DOM on every retry.

LECTURE 18 — File Upload, File Download & Dropdowns

1. File Upload

Method 1: Direct Input File Upload (Most Common)

```
JAVA
// When <input type="file"> is available
page.locator("input[type='file']")
.setInputFiles(Paths.get("src\\test\\resources\\files\\test.pdf"));

// OR using page directly
page.setInputFiles("#uploadFile",
Paths.get("C:/files/resume.pdf"));
```

Method 2: Multiple File Upload

```
JAVA
page.setInputFiles("#uploadFile",
new Path[] {
Paths.get("src\\test\\resources\\files\\file1.pdf"),
Paths.get("src\\test\\resources\\files\\file2.pdf")
});
```

Method 3: Clear Uploaded File

```
JAVA
page.setInputFiles("#uploadFile", new Path[0]);
```

Method 4: FileChooser (When Button Opens File Picker Popup)

```
JAVA
// When clicking a button opens the system file picker dialog
FileChooser fileChooser = page.waitForFileChooser(() -> {
page.click("#uploadBtn"); // This click opens the dialog
});

fileChooser.setFiles(Paths.get("C:/files/resume.pdf"));
```

When to use which:

- `setInputFiles()` → When `<input type="file">` is visible in DOM
- `FileChooser` → When a button opens the OS file picker popup

2. File Download

```

JAVA
// Download triggered by clicking a button
Download download = page.waitForDownload(() -> {
page.locator("text=Download").click(); // Action that starts download
});

// Save the file
Path downloadedFile = Paths.get("downloads", download.suggestedFilename());
download.saveAs(downloadedFile);

// Print the save location
System.out.println("File saved at: " + downloadedFile.toAbsolutePath());

// Download without clicking (using direct URL)
Download download = page.waitForDownload(() -> {
page.evaluate("window.open('https://example.com/file.pdf')");
});

download.saveAs(Path.of("src/test/resources/file.pdf"));

```

Useful Download Methods:

Method	What It Returns
<code>download.path()</code>	Temporary file path
<code>download.suggestedFilename()</code>	Original filename from server
<code>download.saveAs(path)</code>	Save to specific location

3. Dropdowns — 3 Types

Type 1: HTML `<select>` Dropdown (Standard)

```

JAVA
// Select by value attribute
page.selectOption("#country", "IN");

// Select by visible text (label)
page.selectOption("#country", new SelectOption().setLabel("India"));

// Select by index (0-based)
page.selectOption("#country", new SelectOption().setIndex(2));

// Multiple select
page.selectOption("#skills", new String[]{"java", "playwright", "api"});

// Get selected value
String selected = page.locator("#country").inputValue();

// Get all options

```

```
List<String> allOptions = page.locator("#country option").allTextContents();
```

Type 2: Auto-Suggest Dropdown (Google-style)

```
JAVA
// Type text and wait for suggestions
page.fill("#search", "jav");

// Click the correct suggestion
Locator suggestions = page.locator(".suggestions div");
suggestions
    .filter(new Locator.FilterOptions().setHasText("JavaScript"))
    .first()
    .click();

// OR loop through and find the right one
for (int i = 0; i < suggestions.count(); i++) {
    if (suggestions.nth(i).innerText().equals("Java")) {
        suggestions.nth(i).click();
        break;
    }
}
```

Type 3: Custom/Bootstrap Dropdown (div-based)

```
JAVA
// Step 1: Click to open dropdown
page.locator("button#cityDropdown").click();

// Step 2: Click the option
page.getByText("Chennai", new Page.GetByTextOptions().setExact(true)).click();

// OR safer approach – search inside dropdown only
Locator dropdown = page.locator(".options");
dropdown.getByText("Bangalore").click();
```

How to Identify Dropdown Type:

- See `<select>` tag → HTML dropdown → use `selectOption()`
- See `<input>` + suggestions div → Auto-suggest → `fill()` + click
- See `<div>` or `<ul>` + options → Custom → `click()` + `getByText()`

4. Interview Questions — File & Dropdown

Q1: Difference between `setInputFiles()` and `FileChooser` ?

A: `setInputFiles()` works when the file input element is directly in the HTML DOM. `FileChooser` is used when clicking a button opens the OS file picker dialog — Playwright captures the dialog and uploads the file programmatically.

Q2: Why does Playwright not need browser profile for downloads?

A: Playwright handles downloads programmatically using `waitForDownload()` . It captures the download event, holds the file temporarily, and lets you save it anywhere using `saveAs()` . No browser profile needed.

Q3: How do you handle 3 types of dropdowns in Playwright?

A: HTML `<select>` → `selectOption()` . Auto-suggest dropdowns → `fill()` + click suggestion.
Custom/Bootstrap → `click()` to open + `getByText()` to select option.

LECTURE 19 — Multiple Elements & Alert Handling

1. Locator vs Selenium's findElement()

Feature	Selenium	Playwright
Single element	<code>findElement()</code> — immediate search	<code>locator()</code> — deferred search
Multiple elements	<code>findElements()</code> — returns <code>List<WebElement></code>	<code>locator().all()</code> — returns <code>List<Locator></code>
Not found	Throws <code>NoSuchElementException</code>	Creates locator (no error)
Auto-wait	■ No	■ Yes

■ JAVA

```
// Playwright creates a REFERENCE first, searches when action is performed
Locator username = page.locator("#username");
username.fill("venkat"); // Searches and fills here
```

2. Multiple Elements

■ JAVA

```
// Get all matching elements
List<Locator> products = page.locator(".prod-title").all();
for (Locator product : products) {
    System.out.println(product.textContent());
}

// Get all text at once (faster – no loop needed)
List<String> titles = page.locator(".prod-title").allTextContents();
System.out.println(titles); // [Laptop, Phone, Headphones]
```

When to use which:

Situation	Method
Need to click each element	<code>locator().all()</code>
Need only text values	<code>allTextContents()</code>
Specific element by position	<code>locator().nth(0)</code>
First match	<code>locator().first()</code>
Last match	<code>locator().last()</code>

3. Alert Handling — All 5 Types

Alert Type	Part of DOM?	Method	Notes
Simple Alert	■ No	<code>onDialog()</code> + <code>accept()</code>	OK button only
Confirmation Alert	■ No	<code>onDialog()</code> + <code>accept()</code> or <code>dismiss()</code>	OK + Cancel
Prompt Alert	■ No	<code>onDialog()</code> + <code>accept("text")</code>	Text input + OK
Modern Modal/Toast	■ Yes	<code>locator()</code> + <code>click()</code>	Regular HTML element
HTTP Auth Popup	■ No	<code>BrowserContext</code> <code>credentials</code>	Set before navigation

Simple Alert (OK only)

```

// JAVA
// MUST register listener BEFORE clicking (CRITICAL!)
page.onceDialog(dialog -> {
  System.out.println("Message: " + dialog.message());
  dialog.accept(); // Click OK
});
page.locator("#nativeAlert").click();

```

Confirmation Alert (OK + Cancel)

```

// JAVA
// Accept
page.onceDialog(dialog -> {
  dialog.accept(); // Click OK
});
page.click("#nativeConfirm");

// Dismiss
page.onceDialog(dialog -> {
  dialog.dismiss(); // Click Cancel
});
page.click("#nativeConfirm");

```

Prompt Alert (Text + OK)

```

// JAVA
page.onDialog(dialog -> {

```

```
dialog.accept("Playwright Java"); // Types text and clicks OK
});
page.click("#nativePrompt");
```

Modern Modal/Toast

■ JAVA

```
// Toast elements ARE in DOM – use locators directly
page.locator("#modernConfirm").click();
page.locator("#modalOk").click();

// Read toast message
String toastMessage = page.locator("//*[id='toastContainer']/div").innerText();
System.out.println("Toast: " + toastMessage);
```

4. ■ CRITICAL: Register Listener BEFORE Click

■ JAVA

```
// ■ CORRECT – Register first, then click
page.onceDialog(dialog -> { dialog.accept(); });
page.click("#alertButton");

// ■ WRONG – Browser blocks when alert appears, listener never registers
page.click("#alertButton"); // Alert appears, browser freezes!
page.onceDialog(dialog -> { dialog.accept(); }); // Too late!
```

Why? JavaScript alerts BLOCK the browser instantly. If there's no listener ready, the test hangs and times out.

5. onDialog() vs onceDialog()

Feature	<code>page.onDialog()</code>	<code>page.onceDialog()</code>
Type	Permanent listener	One-time listener
Handles	ALL dialogs	Only NEXT dialog
Auto-removed	■ No	■ Yes
Safety	Medium	High ■
Use for	Global handling	Specific test

6. HTTP Authentication Popup


```

// ■ NOT recommended – credentials visible in URL
page.navigate("https://admin:admin@the-internet.herokuapp.com/basic_auth");

// ■ RECOMMENDED – Use BrowserContext credentials
BrowserContext context = browser.newContext(
    new Browser.NewContextOptions()
        .setViewportSize(null)
        .setHttpCredentials("admin", "admin") // Username, Password
);
Page page = context.newPage();
page.navigate("https://the-internet.herokuapp.com/basic_auth");

```

Why not the URL method? Credentials in URL appear in logs, browser history, and reports — security risk!

7. How to Find Toast Message (F8 Freeze Trick)

- Open DevTools (F12)
- Go to **Sources** tab
- Expand **Event Listener Breakpoints**
- Enable **Keyboard** (or Mouse events)
- Perform the action that shows the toast
- Press **F8** to freeze/pause the page
- Go to **Elements** tab and inspect the toast element
- Find its class/ID
- Press Resume ■ button to unfreeze

8. Interview Questions — Alerts

Q1: Why must `onDialog()` be registered before the click?

A: JavaScript alerts block the browser immediately when they appear. No further actions can execute. Playwright must be listening BEFORE the alert appears, otherwise the test hangs and times out.

Q2: How do you handle HTTP Authentication popup in Playwright?

A: HTTP Auth popups are browser-native and not in the DOM. Playwright bypasses them by setting credentials at the **BrowserContext** level using `.setHttpCredentials("username", "password")` before navigating to the page.

Q3: What is the difference between a browser alert and a toast message?

A: Browser alerts (Simple, Confirmation, Prompt) are native browser popups — not in the DOM, cannot be inspected. Toast messages are HTML elements built into the page — they can be inspected in DevTools and handled with regular Playwright locators.

LECTURE 20 — JavaScript Execution in Playwright

1. page.evaluate() — The JavaScript Executor

Playwright equivalent of Selenium's JavascriptExecutor:

```
1 // SeleniUm way (OLD)
2 JavascriptExecutor js = (JavascriptExecutor) driver;
3 js.executeScript("document.getElementById('user').value='admin'");
4
5 // Playwright way (NEW)
6 page.evaluate("document.getElementById('user').value='admin'");
```

2. Common JavaScript Operations

```
1 // Scroll to bottom of page
2 page.evaluate("window.scrollTo(0, document.body.scrollHeight)");
3
4 // Scroll to top
5 page.evaluate("window.scrollTo(0, 0)");
6
7 // Get page title via JavaScript
8 String title = (String) page.evaluate("() => document.title");
9
10 // Set input value directly
11 page.evaluate("document.querySelector('#username').value='admin'");
12
13 // Set date picker (useful when field is readonly)
14 page.evaluate("document.querySelector('#datePicker').value='2026-01-15'");
15
16 // Set readonly field
17 page.evaluate("document.querySelector('#readonly-input').value='Sujal'");
18
19 // Create a JavaScript alert
20 page.evaluate("(name) => alert(name)", "Venkat");
21
22 // Click an element via JavaScript
23 page.evaluate("document.querySelector('#validateInputs').click()");
```

3. locator.evaluate() — Element-Level JavaScript

```
1 // Highlight element with red border (useful for debugging/demos)
```

```

page.locator("#validateInputs")
.evaluate("el => el.style.border='3px solid red'");

// Get text via JavaScript
String text = page.locator("#validateInputs")
.evaluate("el => el.textContent").toString();

// Scroll element into view
page.locator("#validateInputs")
.evaluate("el => el.scrollIntoView()");

// Force click via JavaScript
page.locator("#validateInputs")
.evaluate("el => el.click()");

// Blink/flash an element (demo effect)
page.locator("[type='submit']")
.evaluate("el => setInterval(() => { " +
"el.style.outline = el.style.outline ? '' : '3px solid red'; " +
"}, 300)");

```

4. page.evaluate() vs locator.evaluate()

Feature	page.evaluate()	locator.evaluate()
Scope	Whole page	Specific element
Find element manually	■ Must use JS	■ Not needed
Auto-wait	■ No	■ Yes
Auto-retry	■ No	■ Yes
Reliability	Lower	Higher
Use for	Page-level JS	Element-level JS

5. Preferred Methods Over JavaScript

Task	JavaScript Way	Playwright Preferred Way
Scroll element into view	el.scrollIntoView()	locator.scrollIntoViewIfNeeded()
Get page title	document.title	page.title()
Get input value	element.value	locator.inputValue()
Get text	element.textContent	locator.textContent()

Task	JavaScript Way	Playwright Preferred Way
Click element	<code>element.click()</code>	<code>locator.click()</code>
Scroll page	<code>window.scrollTo()</code>	<code>locator.scrollIntoViewIfNeeded()</code>

Rule: Always prefer Playwright's built-in methods. Use `page.evaluate()` only when native methods cannot handle the scenario.

6. Keyboard & Mouse Actions

Keyboard Actions

```

// JAVA
// Press a key on focused element
page.locator("#search").press("Enter");

// Keyboard shortcuts
page.keyboard().press("Control+A"); // Select all
page.keyboard().press("Control+C"); // Copy
page.keyboard().press("Control+V"); // Paste
page.keyboard().press("Control+X"); // Cut
page.keyboard().press("Delete"); // Delete

// Type text (use this instead of deprecated page.type())
page.keyboard().type("Hello World");

// Type with delay (simulates human typing)
page.keyboard().type("Hello World",
new Keyboard.TypeOptions().setDelay(100)); // 100ms between keys

// Hold key down then up (for text selection)
page.keyboard().down("Shift");
page.keyboard().press("ArrowLeft");
page.keyboard().up("Shift");

// Clear field and type new text
page.locator("#username").press("Control+A");
page.locator("#username").press("Delete");

```

Mouse Actions

```

// JAVA
// Hover over element
page.locator(".nav-dropbtn").hover();

// Right click
page.locator("text='Right Click Me'")
.click(new Locator.ClickOptions().setButton(MouseButton.RIGHT));

```

```
// Double click
page.locator("text=Double Click Me").dblclick();

// Coordinate-based mouse move
page.mouse().move(100, 200);

// Drag and Drop – Method 1 (simplest)
page.dragAndDrop("#source", "#target");

// Drag and Drop – Method 2 (using hover)
Locator source = page.locator("#source");
Locator target = page.locator("#target");
source.hover();
page.mouse().down();
target.hover();
page.mouse().up();

// Drag and Drop – Method 3 (coordinates)
page.mouse().move(200, 300);
page.mouse().down();
page.mouse().move(500, 300);
page.mouse().up();

// Scroll using mouse wheel
page.mouse().wheel(0, 1000); // 0=horizontal, 1000=vertical down
```

Selenium vs Playwright Mouse Actions

Action	Selenium	Playwright
Hover	<code>moveToElement()</code>	<code>hover()</code>
Right-click	<code>contextClick()</code>	<code>setButton(MouseButton.RIGHT)</code>
Double-click	<code>doubleClick()</code>	<code>dblclick()</code>
Drag & Drop	Complex setup	<code>dragAndDrop()</code>

7. Interview Questions — JavaScript & Actions

Q1: What is `page.evaluate()` and when should you use it?

A: `page.evaluate()` executes JavaScript directly inside the browser — similar to Selenium's `JavascriptExecutor`. Use it only when normal Playwright locators cannot handle the scenario, such as setting readonly field values, manipulating the DOM directly, or scrolling.

Q2: Why does Playwright rarely need `JavaScriptExecutor` compared to Selenium?

A: Playwright has built-in auto-waiting, automatic scrolling into view, and better element interaction support. Most scenarios that required JavaScript in Selenium can be handled with Playwright's native methods.

Q3: What is `scrollIntoViewIfNeeded()` and why is it better than JavaScript scrolling?

A: It's a Playwright built-in method that scrolls the element into view only if needed. It has auto-wait and is more reliable than `page.evaluate("el.scrollIntoView()")` .

LECTURE 21 — Screenshots & Video Recording

1. Taking Screenshots

■ JAVA

```
// Full page screenshot (captures entire page including below the fold)
page.screenshot(
    new Page.ScreenshotOptions()
        .setPath(Paths.get("screenshots/fullpage.png"))
        .setFullPage(true)
);

// Viewport screenshot (only what's visible on screen)
page.screenshot(
    new Page.ScreenshotOptions()
        .setPath(Paths.get("screenshots/viewport.png"))
);

// Element screenshot (capture just one element)
Locator element = page.locator("#logo");
element.screenshot(
    new Locator.ScreenshotOptions()
        .setPath(Paths.get("screenshots/logo.png"))
);
```

2. Reusable Screenshot Utility Methods

■ JAVA

```
// Full page screenshot with timestamp (prevents file overwriting)
public static void takeScreenshot(Page page, String name, boolean fullPage) {
    String timestamp = LocalDateTime.now()
        .format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmss"));
    Path path = Paths.get("screenshots/" + name + "_" + timestamp + ".png");
    page.screenshot(new Page.ScreenshotOptions().setPath(path).setFullPage(fullPage));
}

// Element screenshot with timestamp
public static void takeScreenshotElement(Page page, String locator, String name) {
    String timestamp = LocalDateTime.now()
        .format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmss"));
    Path path = Paths.get("screenshots/" + name + "_" + timestamp + ".png");
    page.locator(locator).screenshot(new Locator.ScreenshotOptions().setPath(path));
}
```

```
// Usage
takeScreenshot(page, "AfterLogin", true); // Full page
takeScreenshot(page, "LoginForm", false); // Viewport only
takeScreenshotElement(page, "#logo", "Logo"); // Element only
```

3. Video Recording

■ **JAVA**

```
// Enable video recording at BrowserContext level
BrowserContext context = browser.newContext(
    new Browser.NewContextOptions()
        .setRecordVideoDir(Paths.get("videos/")) // Where to save
        .setRecordVideoSize(1280, 720) // Resolution
);
Page page = context.newPage();
// ... run your test ...
// CRITICAL: Must close context to save the video
context.close();
// Get video path
Path videoPath = page.video().path();
System.out.println("Video saved at: " + videoPath);
browser.close();
playwright.close();
```

■ **CRITICAL:** Video is only saved AFTER `context.close()` . Without closing context, the video file may not be generated or may be incomplete.

4. Screenshot Best Practices

Practice	Reason
Use timestamps in filenames	Prevents overwriting previous screenshots
Capture on failure only	Saves disk space in large test runs
Use meaningful names	Easy to find and understand in reports
Store in <code>screenshots/</code> folder	Organized folder structure

5. Interview Questions — Screenshots & Video

Q1: How do you take a screenshot in Playwright?

A: Use `page.screenshot()` for page screenshots and `locator.screenshot()` for element screenshots. Add `setFullPage(true)` to capture the entire page. Always use timestamps in filenames to avoid overwriting.

Q2: Why must `context.close()` be called for video recording?

A: Video recording happens at the BrowserContext level. The video file is finalized and saved only when the context is closed. Without `context.close()`, the recording may be incomplete or the file may not be created.

Q3: Difference between full-page and viewport screenshot?

A: Viewport screenshot captures only what's visible on screen. Full-page screenshot captures the entire webpage including content below the visible area. Full-page requires `setFullPage(true)`.

PART 2 — QUICK REVISION BEFORE INTERVIEW

CODE

BROWSER FACTORY:

- One class to launch all browsers
- Firefox/WebKit: no `--start-maximized` → use `setViewportSize()`
- Chromium: `--start-maximized` + `setViewportSize(null)`
- Always call `playwright.close()` at end

LOCATORS (Priority Order):

1. `getByRole()` 5. `getByTestId()`
2. `getByLabel()` 6. CSS Selector
3. `getByPlaceholder()` 7. XPath
4. `getByText()`

CRITICAL LOCATOR MISTAKES:

- `.btn primary` ≠ `.btn.primary` (space = descendant, no space = same element)
- XPath index starts at 1, Playwright `nth()` starts at 0

WAITS:

- Auto-wait: `click`, `fill`, `check`, `hover` (automatic)
- Explicit wait: `loaders`, `API data`, `text changes`
- NEVER use `Thread.sleep()`
- Default timeout = 30 seconds

ALERTS (MUST KNOW):

- Register `onDialog()` BEFORE the click that triggers alert
- `onDialog()` = permanent, `onceDialog()` = one-time
- Prompt: `dialog.accept("text")`
- Cancel: `dialog.dismiss()`
- Toast = HTML element, use `locator()`
- HTTP Auth = `BrowserContext.setHttpCredentials()`

FILE OPERATIONS:

- Upload: `setInputFiles()` or `FileChooser`
- Download: `waitForDownload()` + `saveAs()`

DROPDOWNS:

- `<select>`: `selectOption()`
- Auto-suggest: `fill()` + `click suggestion`
- Custom: `click()` + `getByText()`

SCREENSHOTS:

- Viewport: `page.screenshot()`
- Full page: `setFullPage(true)`
- Element: `locator.screenshot()`
- Video: `context.close()` must be called to save

JAVASCRIPT:

- `page.evaluate()` = run JS on whole page
- `locator.evaluate()` = run JS on specific element
- Use JS only when Playwright methods are not enough

Part 2 Complete — Covers Lectures 14 to 21

Continue to Part 3 for Lectures 22 to 30 (Frames, Shadow DOM, POM, TestNG, Cucumber, Reports)

Playwright with Java — Complete Course Notes (Part 3)

Lectures 22 to 30 | Trace Viewer, iFrames, Shadow DOM, POM, TestNG, Reports

LECTURE 21 (Continued) — Playwright Trace Viewer

1. What is Trace Viewer?

Definition: Trace Viewer is Playwright's built-in debugging tool that records EVERYTHING that happened during a test — like a "time machine" for test execution.

Real-World Analogy: Imagine having a flight recorder (black box) for your tests. If something goes wrong, you can replay exactly what happened, step by step.

What it records:

- Every action (click, fill, check)
- Screenshot at every step
- DOM snapshot (what the page HTML looked like)
- Network requests and responses
- Console logs and JavaScript errors
- Timeline of all events

Why Trace Viewer > Regular Video:

Feature	Regular Video	Trace Viewer
Shows what happened	■ Yes	■ Yes
DOM state at each step	■ No	■ Yes
Network requests	■ No	■ Yes
Console logs	■ No	■ Yes
Step-by-step replay	■ No	■ Yes
Inspect element at any moment	■ No	■ Yes

2. Enable Trace Recording

```
■ JAVA  
// STEP 1: Start tracing BEFORE test steps  
context.tracing().start(  
  new Tracing.StartOptions()  
    .setScreenshots(true) // Screenshot at every step  
    .setSnapshots(true) // DOM snapshot at every step  
    .setSources(true) // Include source code info  
);
```

```
// STEP 2: Run your test steps
page.navigate("https://example.com");
page.locator("#username").fill("admin");
page.click("#login");

// STEP 3: Stop and save trace
context.tracing().stop(
  new Tracing.StopOptions()
    .setPath(Paths.get("traces/LoginTest.zip"))
);
```

3. Open Trace Viewer

Method 1: Terminal command

■ BASH

```
npx playwright show-trace traces/LoginTest.zip
```

Method 2: Online (upload ZIP)

Visit: <https://trace.playwright.dev/>

Upload your ZIP file.

4. Best Practice — Trace Only on Failure

■ JAVA

```
context.tracing().start(new Tracing.StartOptions()
    .setScreenshots(true).setSnapshots(true));

try {
    // Test steps
} catch (Exception e) {
    // Save trace only on failure
    context.tracing().stop(new Tracing.StopOptions()
        .setPath(Paths.get("traces/" + testName + ".zip")));
    throw e;
}
```

Why not trace every test? Trace files are large and slow down execution. Record only for failures and flaky tests.

5. Interview Questions — Trace Viewer

Q1: What is Playwright Trace Viewer?

A: Trace Viewer is an advanced debugging tool that records the complete test execution including screenshots, DOM snapshots, network requests, console logs, and timeline. You can replay the test step-by-step to find exactly where and why it failed.

Q2: When should you use Trace Viewer?

A: Use it for debugging failed or flaky tests, especially in CI/CD environments. Record traces only for failures to avoid large file sizes and slower execution.

LECTURE 22 — iFrames & Shadow DOM

1. What is an iFrame?

Definition: An iFrame is a web page embedded INSIDE another web page. Think of it like a TV screen inside a TV screen.

HTML

```
<!-- Main page has an iFrame embedded -->
<iframe src="payment.html"></iframe>
```

Why iFrames are tricky:

- The iFrame has its OWN separate DOM
- You CANNOT directly use `page.locator()` for elements inside an iFrame
- You must TARGET the frame first, then find elements

How to identify an iFrame in DevTools:

- Right-click → Inspect
- Look for `<iframe>` tag in Elements panel

2. FrameLocator (Recommended Method)

JAVA

```
// Basic iFrame using ID or CSS selector
FrameLocator frame = page.frameLocator("#loginFrame");
frame.locator("#username").fill("admin");
frame.locator("#password").fill("secret");
frame.locator("#loginBtn").click();

// iFrame using source URL
page.frameLocator("[src='default.xhtml']").locator("#Click").first().click();

// Nested iFrame (iFrame inside iFrame)
page.frameLocator("iframe:nth-of-type(2)")
    .frameLocator("iframe")
    .locator("button")
    .click();
```

Structure of Nested iFrame:

CODE

```
Main Page
  ■■■ Parent iFrame
    ■■■ Child iFrame
```

```
■■■ Button you want to click
```

3. Frame Object (Older Method)

■ **JAVA**

```
// Get frame by name or ID
Frame frame = page.frame("frameNameOrId");
frame.locator("#username").fill("admin");

// Get frame by URL (useful when no ID)
Frame frame = page.frames().stream()
    .filter(f -> f.url().contains("payment"))
    .findFirst()
    .orElseThrow();
frame.locator("#cardNumber").fill("4111111111111111");

// Index-based frame access
List<Frame> frames = page.frames();
// frames.get(0) = MAIN PAGE always!
// frames.get(1) = First iFrame
// frames.get(2) = Second iFrame
Frame firstIframe = page.frames().get(1);
firstIframe.locator("#username").fill("admin");
```

4. FrameLocator vs Frame Comparison

Feature	Frame	FrameLocator
Type	Actual iframe object	Smart lazy locator
Auto-wait	■ No	■ Yes
Auto-retry	■ No	■ Yes
Can become stale	■ Yes (after page reload)	■ No
Recommended	■ No	■ Yes

5. Switch Back to Main Page

■ **JAVA**

```
// Selenium needs:
driver.switchTo().defaultContent(); // Required in Selenium
```



```
// Playwright – NO switching needed!  
// Just use page.locator() for main page and frameLocator() for iframe  
page.locator("#mainElement").click(); // Main page  
frame.locator("#iframeElement").click(); // Inside iframe
```

6. iFrame Index Rules

■ CODE

```
frames.get(0) → Main page (always!)  
frames.get(1) → First iFrame on page  
frames.get(2) → Second iFrame on page
```

7. What is Shadow DOM?

Definition: Shadow DOM is a HIDDEN DOM attached to an element. It creates a separate, isolated HTML structure inside an element that is protected from outside CSS and JavaScript.

Real-World Analogy: Shadow DOM is like a TV screen with its own remote control — it manages itself independently from everything else.

Common in: Salesforce, ServiceNow, modern web components

■ HTML

```
<login-component>  
#shadow-root (open) ← This is the Shadow DOM  
<input id="username">  
<button id="btn">Login</button>  
</login-component>
```

8. Shadow DOM Types

Type	Automation Access	Notes
Open Shadow DOM	■ Yes — Playwright auto-pierces	Most common
Closed Shadow DOM	■ Cannot access	Very rare

9. Playwright Shadow DOM Handling

```

JAVA
// Playwright automatically pierces Open Shadow DOM
// These work even if element is inside Shadow DOM:
page.locator("#username").fill("admin");

// Using >> operator to traverse shadow boundaries
page.locator("shadow-login >> #user").fill("Venkat");
page.locator("shadow-login >> #btn").click();

// Nested Shadow DOM (shadow inside shadow)
page.locator("shadow-parent >> shadow-child >> #childInput").fill("Playwright");
page.locator("shadow-parent >> shadow-child >> #childBtn").click();

// Shadow DOM inside iFrame
FrameLocator frame = page.frameLocator("#shadowIframe");
frame.locator("[id='user']").last().fill("Sujal");
frame.locator("[id='btn']").last().click();

```

Meaning of >> : Tells Playwright to "enter the Shadow DOM" of the preceding element.

10. iFrame vs Shadow DOM Comparison

Feature	iFrame	Shadow DOM
Type	Separate page inside page	Hidden DOM inside same page
Own DOM	■ Completely separate	■ Hidden but same document
JavaScript context	Separate	Same page
Access method	<code>frameLocator()</code>	Auto-piercing or <code>>></code>
Can inspect in DevTools	■ Yes	■ Yes (if open)
Performance	Slower	Faster

11. Interview Questions — iFrames & Shadow DOM

Q1: What is the recommended way to handle iFrames in Playwright?

A: Use `FrameLocator` with `page.frameLocator()`. It automatically waits and re-locates the frame. Unlike Selenium, you don't need to switch back to the main page — just use `page.locator()` for main page and `frameLocator()` for iframe elements.

Q2: What is Shadow DOM and why does Playwright handle it better than Selenium?

A: Shadow DOM is a hidden DOM that provides encapsulation for web components. Playwright automatically pierces Open Shadow DOM without needing JavaScript workarounds (which Selenium requires). Closed Shadow DOM cannot be accessed by any automation tool.

Q3: What does `frames.get(0)` return?

A: The main page! `frames.get(0)` is always the main page document. `frames.get(1)` is the first iFrame, `get(2)` is the second, etc.

LECTURE 23 — Properties File

1. What is a Properties File?

Definition: A `.properties` file stores configuration data in **key=value** format. Instead of hardcoding values like URLs and passwords in your Java code, you store them in this file.

Benefits:

- No hardcoding in tests
- Easy to change environment (DEV → QA → PROD)
- CI/CD friendly
- One place to update all config

Example `config.properties` :

```
■ PROPERTIES
url=https://swautomationtester-dot.github.io/IshaShopping/
username=admin
password=admin123
browser=chrome
timeout=30000
```

2. Read from Properties File

```
■ JAVA
import java.io.FileInputStream;
import java.util.Properties;

public class ReadProperties {
    public static void main(String[] args) throws IOException {
        // Step 1: Point to the file
        FileInputStream fis = new FileInputStream(
            "src\\main\\resources\\properties\\config.properties");

        // Step 2: Create Properties object
        Properties prop = new Properties();

        // Step 3: Load the file
        prop.load(fis);

        // Step 4: Read values
        String username = prop.getProperty("username"); // "admin"
        String password = prop.getProperty("password"); // "admin123"
        String url = prop.getProperty("url");

        // Use in test
```

```
page.navigate(url);
page.fill("#username", username);
page.fill("#password", password);
}
}
```

3. Write to Properties File

■ **JAVA**

```
// Always load first to preserve existing properties!
FileInputStream fis = new FileInputStream("config.properties");
Properties prop = new Properties();
prop.load(fis); // Read existing data first

// Update or add new value
prop.setProperty("password", "newPassword123");

// Save back to file
FileOutputStream fos = new FileOutputStream("config.properties");
prop.store(fos, "Updated Password"); // Second param is a comment
```

■■ **IMPORTANT:** If you call `setProperty()` and `store()` WITHOUT `load()` first, ALL existing properties will be DELETED and only the new ones will be saved!

4. Interview Questions — Properties File

Q1: Why use a properties file in automation frameworks?

A: Properties files store configuration data (URLs, credentials, browser type, timeouts) separately from test code. This avoids hardcoding, enables easy environment switching, and makes the framework CI/CD friendly.

Q2: What happens if you don't call `prop.load()` before writing?

A: Only the newly added properties will be saved. All existing properties in the file will be deleted and lost.

LECTURE 24 — Apache POI (Excel Operations)

1. What is Apache POI?

Definition: Apache POI is a Java library that allows you to read and write Microsoft Excel files from Java code. Used for Data-Driven Testing — reading test data from Excel.

File Types:

Extension	Class to Use	Excel Version
.xls	HSSFWorkbook	Old (97-2003)
.xlsx	XSSFWorkbook	New (2007+)

Memory Trick:

- H = Historic (old .xls)
- X = XML (new .xlsx)

2. Excel Hierarchy

■ CODE

```
Workbook (Excel File)
  ■■■ Sheet (Tab/Page)
    ■■■ Row (Horizontal line)
      ■■■ Cell (Individual box)
```

3. Maven Dependencies for POI

■ XML

```
<dependency>
<groupId>org.apache.poi</groupId>
<artifactId>poi</artifactId>
<version>5.5.1</version>
</dependency>

<dependency>
<groupId>org.apache.poi</groupId>
<artifactId>poi-ooxml</artifactId>
<version>5.5.1</version>
</dependency>
```

4. Read from Excel

JAVA

```
// Step 1: Open file
FileInputStream fis = new FileInputStream("src\\main\\resources\\excel\\TestData.xlsx");

// Step 2: Load workbook
Workbook workbook = new XSSFWorkbook(fis);

// Step 3: Get sheet
Sheet sheet = workbook.getSheet("Sheet1"); // By name

// Step 4: Get row (0-indexed!)
Row row = sheet.getRow(1); // Row 0 = header, Row 1 = first data row

// Step 5: Get cells
Cell nameCell = row.getCell(0); // First column
Cell cityCell = row.getCell(1); // Second column

// Step 6: Print values
System.out.println("Name: " + nameCell.toString());
System.out.println("City: " + cityCell.toString());

// Step 7: Close
workbook.close();
fis.close();
```

5. Read ALL Rows from Excel (Data-Driven)

JAVA

```
FileInputStream fis = new FileInputStream("LoginData.xlsx");
Workbook workbook = new XSSFWorkbook(fis);
Sheet sheet = workbook.getSheet("Login");

int lastRow = sheet.getLastRowNum(); // Get last row index

// Start from 1 to skip header row
for (int i = 1; i <= lastRow; i++) {
    Row row = sheet.getRow(i);

    String username = row.getCell(0).toString();
    String password = row.getCell(1).toString();

    System.out.println(username + " / " + password);
    // Use in test: loginPage.login(username, password);
}

workbook.close();
fis.close();
```

6. Write to Excel

```
JAVA
// Create new workbook
Workbook workbook = new XSSFWorkbook();

// Create sheet
Sheet sheet = workbook.createSheet("Data");

// Create rows and cells
Row headerRow = sheet.createRow(0);
headerRow.createCell(0).setCellValue("Name");
headerRow.createCell(1).setCellValue("City");

Row dataRow = sheet.createRow(1);
dataRow.createCell(0).setCellValue("Sujal");
dataRow.createCell(1).setCellValue("Kolhapur");

// Save to file
FileOutputStream fos = new FileOutputStream("src\\main\\resources\\excel\\Output.xlsx");
workbook.write(fos);
workbook.close();
fos.close();

System.out.println("Excel file created successfully!");
```

7. Interview Questions — Apache POI

Q1: What classes are used for .xls vs .xlsx files in Apache POI?

A: `HSSFWorkbook` for `.xls` (old format). `XSSFWorkbook` for `.xlsx` (new format). The hierarchy goes: Workbook → Sheet → Row → Cell.

Q2: Why do we start reading from row index 1 instead of 0?

A: Row index 0 is typically the header row (Name, Password, etc.). Test data starts from row 1. We skip row 0 to avoid using header text as test data.

LECTURE 24 (Continued) — Assertions in Playwright

1. What are Assertions?

Definition: Assertions check if the application behaved as expected. They compare the ACTUAL result with the EXPECTED result and determine if the test PASSES or FAILS.

Import needed:

```
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
```

2. Page Assertions

```
// Verify page title
assertThat(page).hasTitle("Dashboard");

// Verify exact URL
assertThat(page).hasURL("https://example.com/dashboard");

// Verify URL contains a pattern (regex)
assertThat(page).hasURL(Pattern.compile(".*dashboard.*"));
```

3. Locator Assertions (Most Used)

```
// Visibility
assertThat(page.locator("#loginBtn")).isVisible();
assertThat(page.locator("#errorMsg")).isHidden();

// Text
assertThat(page.locator("h1")).hasText("Welcome"); // Exact text
assertThat(page.locator(".msg")).containsText("Success"); // Partial text

// Count
assertThat(page.locator(".product")).hasCount(5);

// Attribute
assertThat(page.locator("#email")).hasAttribute("placeholder", "Enter email");

// State
assertThat(page.locator("#submit")).isEnabled();
assertThat(page.locator("#submit")).isDisabled();
```

```

assertThat(page.locator("#name")).isEditable();
assertThat(page.locator("#agree")).isChecked();

// Input value
assertThat(page.locator("#username")).hasValue("admin");

// CSS class
assertThat(page.locator("#submit")).hasClass("btn-primary");

// Negative assertions using .not()
assertThat(page.locator("#error")).not().isVisible();
assertThat(page.locator(".row")).not().hasCount(0);

```

4. Playwright Assertions are Hard Assertions

Definition:

- **Hard Assertion:** If it fails, test STOPS immediately. No more steps run.
- Playwright's built-in `assertThat()` = Hard assertion

For Soft Assertions in Playwright: Use TestNG's `SoftAssert` (covered in TestNG section).

5. Web Tables Handling

```

// JAVA
// Get number of rows
Locator rows = page.locator("#users tbody tr");
int rowCount = rows.count();

// Read all data
for (int i = 0; i < rows.count(); i++) {
    Locator cells = rows.nth(i).locator("td");
    for (int j = 0; j < cells.count(); j++) {
        System.out.print(cells.nth(j).textContent() + " ");
    }
    System.out.println();
}

// Get specific cell (Row 2, Column 3)
String value = page.locator(
    "#users tbody tr:nth-child(2) td:nth-child(3)").textContent();

// Find row by text and click its button
Locator row = page.locator("#users tbody tr",
    new Page.LocatorOptions().setHasText("Kumar"));
row.locator("text=Edit").click();

// Read all values from one column (column 2)
List<String> names = page.locator(

```

```
#users tbody tr td:nth-child(2)").allTextContents();  
  
// Verify value exists in table  
assertThat(page.locator("#users tbody tr")).containsText("Ravi");  
  
// Reusable method to get any cell  
public String getCellValue(String tableSelector, int row, int col) {  
    return page.locator(tableSelector +  
        " tbody tr:nth-child(" + row + ") td:nth-child(" + col + ")")  
        .textContent();  
}
```

6. Interview Questions — Assertions

Q1: What is the difference between Hard and Soft assertions?

A: Hard assertions stop test execution immediately when they fail. Soft assertions (TestNG `SoftAssert`) continue execution after failure and report all failures together at `assertAll()` .

Q2: What is auto-waiting in Playwright assertions?

A: Playwright assertions automatically wait until the expected condition is met or the timeout is reached. You don't need to add explicit waits before assertions.

LECTURE 25 — Extent Reports

1. What is Extent Reports?

Definition: Extent Reports generates beautiful HTML test execution reports that show PASS/FAIL status, screenshots, logs, charts, and summary — much better than plain console output.

What it shows:

- Test case names and status (PASS/FAIL/SKIP)
- Step-by-step execution logs
- Failure screenshots
- Execution time
- Categories (Smoke/Regression)
- System info (OS, Browser, Tester)
- Dashboard charts

2. Maven Dependency

■ XML

```
<dependency>
<groupId>com.aventstack</groupId>
<artifactId>extentreports</artifactId>
<version>5.1.1</version>
</dependency>
```

3. Three Core Components

ExtentSparkReporter → Creates the HTML report file

ExtentReports → Manages all test results

ExtentTest → Represents one test case

■ JAVA

```
// 1. Create HTML reporter (defines output file)
ExtentSparkReporter reporter = new ExtentSparkReporter("reports/report.html");

// 2. Configure report appearance
reporter.config().setDocumentTitle("Test Report");
reporter.config().setReportName("Login Validation");
reporter.config().setTheme(Theme.DARK);
```

```
// 3. Create main reports manager
ExtentReports extent = new ExtentReports();

// 4. Connect reporter to extent
extent.attachReporter(reporter);

// 5. Add system info
extent.setSystemInfo("Application", "Swag Labs");
extent.setSystemInfo("Browser", "Chrome");
extent.setSystemInfo("Environment", "QA");
extent.setSystemInfo("OS", System.getProperty("os.name"));
extent.setSystemInfo("Tester", "Sujal");

// 6. Create a test entry
ExtentTest test = extent.createTest("Login Test")
    .assignAuthor("Sujal Khot")
    .assignCategory("Smoke")
    .assignDevice("Chrome");

// 7. Log execution steps
test.info("Browser launched");
test.info("Navigated to login page");
test.info("Entered username");
test.info("Clicked login button");

// 8. Log test result
test.pass("Login successful");
// OR
test.fail("Login failed - " + e.getMessage());

// 9. Attach screenshot
String screenshotPath = "screenshots/login.png";
page.screenshot(new Page.ScreenshotOptions().setPath(Paths.get(screenshotPath)));
test.addScreenCaptureFromPath(screenshotPath);

// 10. MANDATORY - Generate final HTML report
extent.flush();
```

4. Log Status Types

Method	Color	Meaning
<code>test.pass("message")</code>	■ Green	Test step succeeded
<code>test.fail("message")</code>	■ Red	Test step failed
<code>test.info("message")</code>	■ Blue	Informational log
<code>test.warning("message")</code>	■ Orange	Warning (not a failure)
<code>test.skip("message")</code>	■ Yellow	Test skipped

5. Best Practice Pattern with try-catch

```
■ JAVA
test = extent.createTest("Login Test").assignCategory("Smoke");
test.info("Browser launched");

try {
    page.navigate("https://example.com");
    test.info("Navigated to site");
    page.fill("#username", "admin");
    test.info("Entered username");
    page.click("#login");
    test.info("Clicked login");
    assertThat(page).hasURL("dashboard");
    test.pass("Login successful!");
} catch (Exception e) {
    // Capture screenshot on failure
    String path = "screenshots/failure_" + System.currentTimeMillis() + ".png";
    page.screenshot(new Page.ScreenshotOptions().setPath(Paths.get(path)));
    test.addScreenCaptureFromPath(path);
    test.fail("Test failed: " + e.getMessage());
    throw e; // Re-throw so TestNG marks the test as failed
}
```

6. ■ extent.flush() is MANDATORY

Without `extent.flush()` :

- HTML report may be empty
- Logs will not appear in report
- Report may not be saved to disk

Always call `extent.flush()` at the end of all tests (in `@AfterSuite`).

7. Interview Questions — Extent Reports

Q1: What are the three core components of Extent Reports?

A: `ExtentSparkReporter` creates the HTML file and defines its appearance. `ExtentReports` is the main manager that collects all test results and logs. `ExtentTest` represents one test case. They are connected using `extent.attachReporter(reporter)` .

Q2: What happens if you forget `extent.flush()` ?

A: The HTML report is not generated or is empty. All logged information is lost. `flush()` is mandatory — it writes all collected data to the HTML file.

LECTURE 26 — Page Object Model (POM)

1. What is POM?

Definition: Page Object Model is a design pattern where each web page of your application is represented as a separate Java class. Each class contains the page's locators and the actions you can perform on that page.

Simple Rule:

- `LoginPage.java` → Stores login form locators + login actions
- `DashboardPage.java` → Stores dashboard locators + dashboard actions
- `LoginTest.java` → Calls page methods and has assertions

2. Without POM vs With POM

Without POM (Bad — everything in test):

```
// JAVA
// Test has locators + actions + assertions mixed together – messy!
page.fill("#username", "admin");
page.fill("#password", "admin123");
page.click("#loginBtn");
page.click("#menu");
page.click("#logout");
assertThat(page).hasURL("login");
```

With POM (Good — clean separation):

```
// JAVA
LoginPage loginPage = new LoginPage(page);
DashboardPage dashboard = new DashboardPage(page);
loginPage.login("admin", "admin123");
dashboard.logout();
assertThat(page).hasURL("login"); // Only assertion in test
```

3. Page Class Structure

```
// JAVA
public class LoginPage {
    private Page page; // Playwright Page object
    // Locators stored as private fields
```



```
private Locator txtUsername;
private Locator txtPassword;
private Locator btnLogin;

// Constructor – receives the Page object
public LoginPage(Page page) {
    this.page = page;
    // Initialize locators here
    txtUsername = page.locator("#username");
    txtPassword = page.locator("#password");
    btnLogin = page.locator("#loginBtn");
}

// Individual action methods
public void enterUsername(String username) {
    txtUsername.fill(username);
}

public void enterPassword(String password) {
    txtPassword.fill(password);
}

public void clickLogin() {
    btnLogin.click();
}

// Business method (combines actions)
public void login(String username, String password) {
    enterUsername(username);
    enterPassword(password);
    clickLogin();
}

// Verification method (can return value for assertion)
public boolean isErrorVisible() {
    return page.locator(".error-message").isVisible();
}
}
```

4. Test Class Structure

■ JAVA

```
public class LoginTest {
    Playwright playwright;
    Browser browser;
    Page page;

    @BeforeMethod
    public void setup() {
```

```

playwright = Playwright.create();
browser = playwright.chromium().launch(
    new BrowserType.LaunchOptions().setHeadless(false));
page = browser.newPage();
}

@Test
public void testSuccessfulLogin() {
    // Create page object
    LoginPage loginPage = new LoginPage(page);
    // Navigate (can be in page class too)
    page.navigate("https://example.com");
    // Call page method (not locators directly!)
    loginPage.login("admin", "admin123");

    // Assertions only here
    assertThat(page).hasURL("https://example.com/dashboard");
}

@AfterMethod
public void teardown() {
    browser.close();
    playwright.close();
}
}

```

5. Recommended POM Framework Structure

■ CODE

```

src/
  ■■■ main/java/
    ■ ■■■ base/
      ■ ■ ■■■ BasePage.java ← Common methods (click, fill, scroll)
    ■ ■■■ pages/
      ■ ■ ■■■ LoginPage.java ← Login page locators + actions
      ■ ■ ■■■ DashboardPage.java ← Dashboard locators + actions
      ■ ■ ■■■ ProductPage.java ← Product page locators + actions
    ■ ■■■ utils/
      ■ ■■■ BrowserFactory.java ← Browser setup
      ■ ■■■ ConfigManager.java ← Read properties file
      ■ ■■■ ExcelUtils.java ← Read Excel test data
    ■
  ■■■ main/resources/
    ■ ■■■ properties/
      ■ ■■■ config.properties ← URL, browser, credentials
    ■

```

```

■■■ test/java/
■■■ tests/
■■■ LoginTest.java ← Test assertions + flow
■■■ CartTest.java ← Cart test assertions

```

6. Page Class Responsibilities

POM Class	Should Contain	Should NOT Contain
Page Class	Locators, Actions, Page Methods	Assertions, @Test, Test Logic
Test Class	Assertions, Test Scenarios	Locators, Raw Actions
Utils	Helper methods	Business logic

7. POM Benefits

Benefit	Explanation
Reusability	<code>loginPage.login()</code> used in 50 tests
Easy Maintenance	If <code>#loginBtn</code> changes to <code>#signIn</code> , change only <code>LoginPage.java</code>
Readability	Test reads like English — <code>loginPage.login("admin", "pass")</code>
No Duplication	Locators defined once, used everywhere
Team Friendly	BA can understand tests without Java knowledge

8. Interview Questions — POM

Q1: What is Page Object Model and why do we use it?

A: POM is a design pattern where each web page is a separate Java class with its locators and actions. Tests only call page methods and contain assertions. This avoids code duplication, makes maintenance easy (one locator change = one place to update), and makes tests readable.

Q2: What should a page class NOT contain?

A: Page classes should NOT contain assertions (`assertThat()`), `@Test` methods, or test scenarios. These belong in test classes. Page classes should only have locators and reusable action methods.

Q3: Why do page classes have a constructor that takes `Page page` ?

A: The Page object from Playwright is what allows you to interact with the browser. By passing it through the constructor, the page class can use it to define locators and perform actions. All page classes share the same Page object (same browser tab).

LECTURE 27 & 28 — TestNG Framework

1. What is TestNG?

Definition: TestNG (Testing Next Generation) is a Java testing framework that helps you organize, run, and report your automated tests. In Playwright automation, TestNG acts as the test runner.

Without TestNG:

- Run `main()` methods manually one by one
- No automatic reports
- No grouping or parallel execution
- Not suitable for large projects

With TestNG:

- One-click execution of all tests
- Automatic reports generated
- Groups (Smoke/Regression)
- Priorities, dependencies
- Parallel execution
- Retry failed tests

2. TestNG Annotations (Most Important)

Annotation	Runs When	How Many Times
<code>@BeforeSuite</code>	Before all tests in suite	Once
<code>@AfterSuite</code>	After all tests in suite	Once
<code>@BeforeClass</code>	Before all tests in class	Once per class
<code>@AfterClass</code>	After all tests in class	Once per class
<code>@BeforeMethod</code>	Before EACH test method	Before every <code>@Test</code>
<code>@AfterMethod</code>	After EACH test method	After every <code>@Test</code>
<code>@Test</code>	Actual test method	Each test

Execution Order:

■ CODE

```
@BeforeSuite → @BeforeClass → @BeforeMethod → @Test → @AfterMethod → @AfterClass → @AfterSuite
```

3. Playwright + TestNG Setup Pattern

```
JAVA
public class BaseTest {
    protected Playwright playwright;
    protected Browser browser;
    protected BrowserContext context;
    protected Page page;

    @BeforeClass // Browser launch – once per class
    public void setupBrowser() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch(
            new BrowserType.LaunchOptions().setHeadless(false));
    }

    @BeforeMethod // New page – before every test
    public void createPage() {
        context = browser.newContext();
        page = context.newPage();
    }

    @AfterMethod(alwaysRun = true) // Always close page
    public void closePage() {
        context.close();
    }

    @AfterClass(alwaysRun = true) // Always close browser
    public void closeBrowser() {
        browser.close();
        playwright.close();
    }
}
```

What `alwaysRun = true` does: Forces the method to run even if the test before it failed. Ensures cleanup always happens — browser closes properly, reports get generated.

4. TestNG Features

Priority

```
JAVA
@Test(priority = 0) // Runs first
public void launchBrowser() {}

@Test(priority = 1) // Runs second
public void login() {}
```

```
@Test(priority = 2) // Runs third
public void addToCart() {}
```

Rule: Lower priority number = runs first. Default priority = 0.

Dependency

```
JAVA
@Test
public void login() {}

@Test(dependsOnMethods = {"login"}) // Runs ONLY if login() passes
public void dashboard() {}

@Test(dependsOnMethods = {"dashboard"}) // Runs only if dashboard() passes
public void checkout() {}
```

If `login()` fails → `dashboard()` is SKIPPED.

Groups

```
JAVA
@Test(groups = {"Smoke"})
public void loginTest() {}

@Test(groups = {"Regression"})
public void paymentTest() {}

@Test(groups = {"Smoke", "Regression"}) // Belongs to multiple groups
public void homePageTest() {}

@Test(enabled = false) // Skip this test
public void brokenTest() {}
```

testng.xml to run specific group:

```
XML
<groups>
<run>
<include name="Smoke"/>
</run>
</groups>
```

Parameters (from testng.xml)

```
XML
<!-- testng.xml -->
<parameter name="UserName" value="admin"/>
<parameter name="Password" value="admin123"/>
```

```
JAVA
@Test
@Parameters({"UserName", "Password"})
public void loginTest(String user, String pass) {
```

```
loginPage.login(user, pass);
}
```

DataProvider (Multiple Data Sets)

■ **JAVA**

```
@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        {"admin", "admin123"}, // Data set 1
        {"user", "user123"} // Data set 2
    };
}

@Test(dataProvider = "loginData")
public void loginTest(String username, String password) {
    // Runs TWICE – once for each data set
    loginPage.login(username, password);
}
```

5. Parallel Execution in testng.xml

■ **XML**

```
<suite name="Suite" parallel="tests" thread-count="3">
  <test name="ChromeTests">
    <parameter name="browser" value="chrome"/>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
  <test name="FirefoxTests">
    <parameter name="browser" value="firefox"/>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
  <test name="EdgeTests">
    <parameter name="browser" value="edge"/>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
</suite>
```

Parallel modes:

Mode	What Runs in Parallel
<code>parallel="methods"</code>	Test methods simultaneously
<code>parallel="classes"</code>	Test classes simultaneously

Mode	What Runs in Parallel
<code>parallel="tests"</code>	<code><test></code> tags simultaneously (most common)

6. Re-Run Failed Tests

Method 1: Manual — testng-failed.xml

After a run, TestNG creates `test-output/testng-failed.xml` with only failed tests.

Right-click → Run As → TestNG Suite → selects failed tests only.

Method 2: Automatic — Retry Analyzer

```

JAVA
// RetryAnalyzer class
public class RetryAnalyzer implements IRetryAnalyzer {
    int counter = 0;
    int retryLimit = 1; // Retry 1 time

    @Override
    public boolean retry(ITestResult result) {
        if (counter < retryLimit) {
            counter++;
            return true; // true = retry
        }
        return false; // false = don't retry again
    }
}

// AnnotationTransformer – applies retry to ALL tests automatically
public class AnnotationTransformer implements IAnnotationTransformer {
    @Override
    public void transform(ITestAnnotation annotation, Class testClass,
        Constructor testConstructor, Method testMethod) {
        annotation.setRetryAnalyzer(RetryAnalyzer.class);
    }
}

```

Add to testng.xml:

```

XML
<listeners>
<listener class-name="utils.AnnotationTransformer"/>
</listeners>

```

7. TestNG Assertions

Hard Assertions (org.testng.Assert)

■ JAVA

```
Assert.assertEquals("Dashboard", "Dashboard"); // PASS
Assert.assertEquals("Admin", "Dashboard"); // FAIL – stops immediately
Assert.assertTrue(10 > 5); // PASS
Assert.assertFalse(5 > 10); // PASS
Assert.assertNotNull(page); // PASS
Assert.assertNull(null); // PASS
Assert.assertNotEquals("a", "b"); // PASS
Assert.fail("Forced fail"); // Always FAIL
```

Soft Assertions (org.testng.asserts.SoftAssert)

■ JAVA

```
SoftAssert sa = new SoftAssert();
sa.assertEquals("Login", "Login"); // Check 1 – continues even if fails
sa.assertEquals("Wrong", "Right"); // Check 2 – FAILS but continues
sa.assertTrue(5 > 2); // Check 3 – PASS
sa.assertFalse(5 < 2); // Check 4 – PASS
sa.assertAll(); // MANDATORY – reports all failures here
```

Hard vs Soft:

Feature	Hard Assert	Soft Assert
On failure	Test STOPS	Test CONTINUES
Reports failures	At point of failure	At <code>assertAll()</code>
Class	<code>Assert</code>	<code>SoftAssert</code>
Use case	Critical checks	Validate multiple UI elements

8. Interview Questions — TestNG

Q1: What is the difference between @BeforeClass and @BeforeMethod in Playwright?

A: `@BeforeClass` runs once per class — ideal for browser launch. `@BeforeMethod` runs before every `@Test` — ideal for creating a new BrowserContext and Page for each test (fresh state).

Q2: What is `alwaysRun = true` ?

A: It forces a configuration method to execute even if a previous test or configuration method fails. Used with `@AfterMethod` and `@AfterSuite` to ensure cleanup always happens.

Q3: What is the difference between @Parameters and @DataProvider ?

A: `@Parameters` passes one set of data from testng.xml — test runs once. `@DataProvider` passes multiple sets of data from Java code — test runs multiple times, once per data set.

Q4: What is IRetryAnalyzer?

A: It's a TestNG interface for automatically re-running failed tests. The `retry()` method returns `true` to retry, `false` to stop. Combined with `IAnnotationTransformer`, it applies automatically to all tests.

LECTURE 30 — Cucumber BDD Framework

1. What is Cucumber?

Definition: Cucumber is a BDD (Behavior Driven Development) framework that lets you write tests in plain English that both technical and non-technical people can understand.

What is BDD? BDD = Behavior Driven Development. Tests are written from the user's perspective describing the behavior of the application.

2. Gherkin Language

Cucumber uses **Gherkin** — a simple English-like language with special keywords:

Keyword	Meaning	Example
Feature	What functionality we're testing	Feature: Login Functionality
Scenario	One specific test case	Scenario: Valid Login
Given	Pre-condition (starting state)	Given User is on Login Page
When	Action performed	When User enters username and password
Then	Expected result	Then User should see Dashboard
And	Additional step	And User clicks Login button
But	Negative condition	But Error message should not appear

3. Feature File Example

■ GHERKIN
Feature: Login Functionality
Scenario: Valid Login
Given User is on Login Page
When User enters username "admin"
And User enters password "admin123"
And User clicks Login button
Then User should see Dashboard page

Scenario: Invalid Login
Given User is on Login Page
When User enters username "wrong"
And User enters password "wrong"
And User clicks Login button
Then User should see error message

4. Cucumber Framework Structure

■ CODE

src/test/java/
■■■ stepdefinitions/
■ ■■■ LoginSteps.java ← Java code for each Gherkin step
■■■ runners/
■ ■■■ TestRunner.java ← Runs the feature files
■■■ hooks/
■■■ Hooks.java ← @Before (setup) and @After (teardown)
src/test/resources/
■■■ features/
■■■ login.feature ← Gherkin scenarios

5. Step Definition Example

■ JAVA

// LoginSteps.java
public class LoginSteps {
Page page;
@Given("User is on Login Page")
public void userIsOnLoginPage() {
// Launch browser and navigate
page.navigate("https://example.com");
}
@When("User enters username {string}")
public void userEntersUsername(String username) {
page.fill("#username", username);
}
@When("User enters password {string}")
public void userEntersPassword(String password) {
page.fill("#password", password);
}

```

@When("User clicks Login button")
public void userClicksLogin() {
    page.click("#loginBtn");
}

@Then("User should see Dashboard page")
public void userShouldSeeDashboard() {
    assertThat(page).hasURL("dashboard");
}
}

```

6. TestRunner Example

```

// JAVA
@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepdefinitions",
    plugin = {
        "pretty",
        "html:target/cucumber-reports/cucumber.html",
        "json:target/cucumber-reports/cucumber.json"
    }
)
public class TestRunner {}

```

Plugin Options:

Plugin	Output	Use
<code>pretty</code>	Console	Formatted readable output
<code>html:path</code>	HTML file	Browser-friendly report
<code>json:path</code>	JSON file	For Allure/Extent integration
<code>junit:path</code>	XML file	For CI/CD (Jenkins)

7. Interview Questions — Cucumber

Q1: What is BDD and how is Cucumber related to it?

A: BDD (Behavior Driven Development) focuses on application behavior from a user's perspective. Cucumber implements BDD by allowing tests to be written in Gherkin (plain English) that both developers and business people can understand.

Q2: What is the difference between Feature file and Step Definitions?

A: Feature file contains the Gherkin scenarios in plain English (what to test). Step Definition file contains the Java code that automates those scenarios (how to test).

FINAL SECTION — COMPLETE QUICK REVISION

ONE-DAY INTERVIEW REVISION

Most Important 20% Concepts (Covers 80% of Interview Questions)

■ CODE

1. PLAYWRIGHT BASICS:
 - Created by Microsoft, 2020
 - WebSocket connection (vs HTTP in Selenium)
 - PBP: Playwright → Browser → Page
 - Default timeout: 30 seconds
2. LOCATORS (Priority Order):
 1. getByRole() 2. getByLabel() 3. getByPlaceholder()
 4. getByText() 5. getByTestId() 6. CSS 7. XPath
3. AUTO-WAIT vs THREAD.SLEEP():
 - Auto-wait: built into every action (click, fill, check)
 - Thread.sleep(): NEVER use – always waits full time
 - Explicit wait: for loaders, API data, text changes
4. FILL() vs TYPE():
 - fill() = clears + types (use for most cases)
 - type() = appends without clearing
5. ALERTS:
 - Register onDialog() BEFORE the click (critical!)
 - onceDialog() = one-time, safer than onDialog()
 - HTTP Auth = BrowserContext.setHttpCredentials()
 - Toast = use locator() directly
6. IFRAMES:
 - Use frameLocator() (recommended)
 - frames.get(0) = main page always
 - No need to switchTo().defaultContent() in Playwright
7. SHADOW DOM:
 - Playwright auto-pierces Open Shadow DOM
 - >> operator to traverse shadow boundaries
 - Closed Shadow DOM = cannot access
8. HARD vs SOFT ASSERTION:
 - Hard: test stops immediately on failure
 - Soft: continues, reports all failures at assertAll()
9. POM:
 - Page class = locators + actions (NO assertions)
 - Test class = assertions + test logic (NO locators)

- Constructor receives Page object
10. TESTNG:
- @BeforeClass = browser launch
- @BeforeMethod = new page for each test
- @AfterMethod(alwaysRun=true) = close page always
- @DataProvider = multiple data sets
- @Parameters = single data from XML
11. DROPDOWNS:
- <select>: selectOption()
- Auto-suggest: fill() + click suggestion
- Custom/Bootstrap: click() + getByText()
12. FILE OPERATIONS:
- Upload (input visible): setInputFiles()
- Upload (popup): FileChooser
- Download: waitForDownload() + saveAs()
13. EXTENT REPORTS:
- 3 components: ExtentSparkReporter, ExtentReports, ExtentTest
- extent.flush() is MANDATORY (generates HTML)
- Screenshot on failure: test.addScreenCaptureFromPath()
14. TRACE VIEWER:
- context.tracing().start() before test
- context.tracing().stop() after test
- Open: npx playwright show-trace trace.zip
15. VIDEO RECORDING:
- Enable: setRecordVideoDir() on BrowserContext
- Saved ONLY after context.close()

COMMON MISTAKES TO AVOID

Mistake	Correct Approach
Using <code>Thread.sleep()</code>	Use Playwright auto-wait or <code>locator.waitFor()</code>
<code>.btn primary</code> (space)	<code>.btn.primary</code> (no space = same element)
XPath index starting at 0	XPath starts at 1: <code>(//input)[1]</code>
Playwright <code>.nth()</code> starting at 1	<code>.nth(0)</code> is the first element
<code>initBroswer</code> (typo)	<code>initBrowser</code> (correct spelling)
Forgetting <code>extent.flush()</code>	Always call at end — report won't generate
Forgetting <code>context.close()</code> for video	Video file won't save without it
Registering <code>onDialog()</code> after click	Must register BEFORE the click

Mistake	Correct Approach
Using <code>..</code> XPath traversal	Use parent-child XPath for readability
No <code>prop.load()</code> before writing	Always load first or existing data is deleted
Using <code>ElementHandle</code> for dynamic elements	Use <code>Locator</code> — it re-checks DOM every time

INTERVIEW ANSWERS IN ONE LINE

Question	One-Line Answer
What is Playwright?	Microsoft's 2020 WebSocket-based automation framework with built-in auto-waiting
Auto-wait?	Playwright automatically waits for elements to be visible, enabled, and stable before acting
POM?	Each web page = one Java class containing locators and actions (no assertions)
Hard vs Soft assertion?	Hard = stops on failure; Soft = continues, reports at <code>assertAll()</code>
iFrame handling?	Use <code>frameLocator()</code> — auto-waits and re-locates, no <code>switchTo</code> needed
Shadow DOM?	Playwright auto-pierces Open Shadow DOM; use <code>>></code> for explicit traversal
Alert handling?	Register <code>onDialog()</code> BEFORE the action that triggers the alert
HTTP Auth popup?	Set credentials on <code>BrowserContext</code> using <code>.setHttpCredentials()</code>
<code>fill()</code> vs <code>type()</code> ?	<code>fill()</code> clears and types; <code>type()</code> appends without clearing
<code>BrowserContext</code> ?	Isolated browser session — like an incognito window

All 3 Parts Complete — Covers Lectures 0 to 30

Good luck with your interviews!

Good Luck With Your Interview!

- ✓ These notes cover the complete Playwright with Java automation testing course.
- ✓ All concepts are explained in beginner-friendly language with real-world examples.
- ✓ The Quick Revision section at the end covers the 20% concepts that answer 80% of interview questions.
- ✓ Review the 'Common Mistakes to Avoid' table before every interview.